Hybrid Quantum–Classical Autonomy for Launch Vehicles

From Six-Degree-of-Freedom Simulation to
Flight-Ready Guidance, Navigation, and Control

Draft — Chapters 1–2

Chapter 1

Introduction: Autonomous Launch Vehicles and the Hybrid Classical–Quantum Software Stack

> *A rocket does not fly the trajectory you planned. It flies the trajectory that physics, wind, engine variation and sensor noise allow. The software's job is to close that gap, many times per second, without anyone on the ground touching it.*

This chapter sets the scene for the whole book. We will not build anything complete yet. Instead, we meet each of the five ideas the book rests on, and for each one we write down the small amount of mathematics that makes it precise:

1. **Simulation** — a launch vehicle is a dynamical system $\dot{x} = f(x, u, t)$, and we test everything on a computer before it flies (Section 1.2).

2. **Feedback** — measuring the error and correcting it makes a system tolerant of things we did not model (Section 1.3).

3. **Estimation** — the vehicle never knows its true state; it keeps a best guess and a measure of how wrong that guess might be (Section 1.4).

4. **Optimization** — guidance is the problem of choosing the best control history that obeys the physics and the hardware limits (Section 1.5).

5. **Quantum computation** — a different model of computing that may help with some hard subproblems, and whose limits we must state honestly (Section 1.6).

We then show how these pieces fit into the four-phase architecture of the book (Section 1.7), fix our notation (Section 1.8), and give a road map of the chapters that follow (Section 1.9).

Take your time with the worked examples. Each one uses real numbers, so you can check every step with a calculator.

1.1 The problem we are solving

An *autonomous launch vehicle* must do three things without a human in the loop:

Ascent. Lift off, climb through the atmosphere and reach a target orbit or a target state (position, velocity, time) within tight tolerances.

Guided descent. For reusable vehicles, return through the atmosphere and land at a chosen point with almost zero velocity.

Fault recovery. Notice when something has gone wrong — a failed sensor, an engine that has shut down, a trajectory drifting off-nominal — and change its behaviour so the mission can still succeed, or at least end safely.

All three are done by *flight software*. That software is organised into the classic guidance, navigation and control (GNC) loop shown in Fig. 1.1:

- **Navigation** answers *"Where am I, how fast am I going, and which way am I pointing?"*
- **Guidance** answers *"Where should I be going from here?"*
- **Control** answers *"How do I move the engine nozzle or fins to get there?"*

A fourth layer, **fault detection, isolation and recovery (FDIR)**, watches the other three and steps in when the numbers stop making sense.

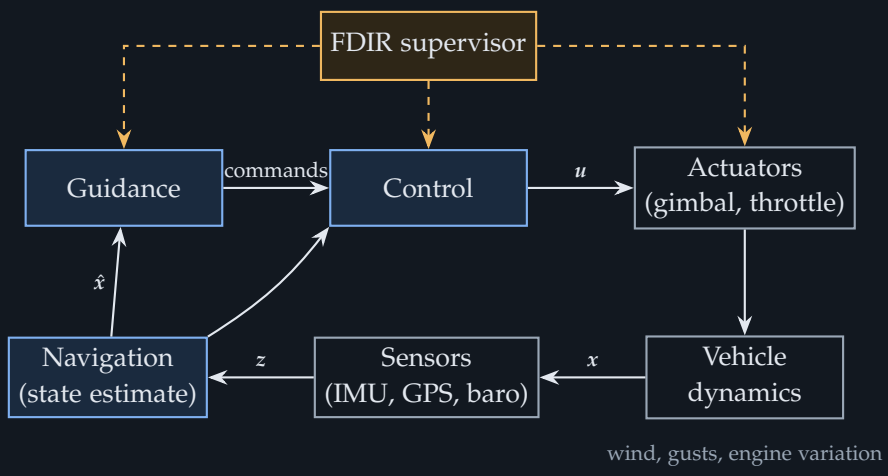


Figure 1.1: The guidance, navigation and control loop. Blue blocks are flight software, white blocks are physical hardware (simulated in the early phases of this book), and the amber supervisor is the fault-management layer. x is the true state, z the measurements, $\hat{x}$ the estimated state and u the actuator commands.

> **Key idea.** Notice that guidance and control only ever see $\hat{x}$, the *estimate*, never the true state x. Every decision the vehicle makes is a decision under uncertainty. This one fact is why estimation, robustness and fault detection take up so much of this book.

1.2 Simulation first: virtual engineering

Rockets are expensive, and a failed flight usually destroys the hardware. So modern aerospace development follows a *simulation-first* workflow, often called *virtual engineering*: build a mathematical model of the vehicle and its environment, write the flight software against that model, and only move to real hardware once the software has passed thousands of simulated flights. Zipfel's text calls this a progression from concept, through design and test, to operation, with simulation used at every stage. (source: Modelling and Simulation of Aerospace Vehicle Dynamics Authored by Zipfel)

1.2.1 The vehicle as a dynamical system

Everything we know about the vehicle at one instant is collected into a *state vector* $\boldsymbol{x}(t)$. For a rigid launch vehicle a natural choice is

$$\boldsymbol{x} = \begin{bmatrix} \boldsymbol{r} \\ \boldsymbol{v} \\ \boldsymbol{q} \\ \boldsymbol{\omega} \\ m \end{bmatrix} \in \mathbb{R}^{14}, \qquad \begin{aligned} &\boldsymbol{r} \in \mathbb{R}^3 && \text{position} \\ &\boldsymbol{v} \in \mathbb{R}^3 && \text{velocity} \\ &\boldsymbol{q} \in \mathbb{R}^4 && \text{attitude quaternion} \\ &\boldsymbol{\omega} \in \mathbb{R}^3 && \text{angular velocity} \\ &m \in \mathbb{R} && \text{mass.} \end{aligned} \tag{1.1}$$

That is $3+3+4+3+1 = 14$ numbers. The three translational and three rotational degrees of freedom give the name *six-degree-of-freedom (6-DOF) simulation*.

The laws of physics tell us how fast each of these numbers changes. Writing $\boldsymbol{u}$ for the inputs we control (engine gimbal angles, throttle) and t for time, the whole simulator is one equation:

$$\boxed{\dot{\boldsymbol{x}}(t) = \boldsymbol{f}\big(\boldsymbol{x}(t),\, \boldsymbol{u}(t),\, t\big), \qquad \boldsymbol{x}(t_0) = \boldsymbol{x}_0.} \tag{1.2}$$

Building a good simulator means writing down a good $\boldsymbol{f}$. Running a simulation means solving (1.2) numerically. Let us look at the most important parts of $\boldsymbol{f}$ now, at a level of detail that Part I of the book will fill in fully.

1.2.2 Translation with changing mass: the rocket equation

A rocket is a body that throws away part of itself to move. Newton's second law in the form $\boldsymbol{F} = m\boldsymbol{a}$ assumes constant mass, so we have to be careful. We use conservation of momentum over a short time $\mathrm{d}t$, in one dimension for clarity.

At time t the rocket has mass m and velocity v. During $\mathrm{d}t$ it ejects a small mass of propellant, so its mass changes by $\mathrm{d}m$ (a *negative* number). The ejected mass $-\,\mathrm{d}m$ leaves with exhaust speed v_e relative to the rocket, so its velocity in the inertial frame is $v - v_e$. An external force F (gravity, drag) acts during $\mathrm{d}t$. Momentum balance gives

$$\underbrace{(m+\mathrm{d}m)(v+\mathrm{d}v)}_{\text{rocket after}} + \underbrace{(-\,\mathrm{d}m)(v-v_e)}_{\text{exhaust}} = \underbrace{mv}_{\text{before}} + F\,\mathrm{d}t. \tag{1.3}$$

Expanding and cancelling mv and $v\,\mathrm{d}m$:

$$m\,\mathrm{d}v + \mathrm{d}m\,\mathrm{d}v + v_e\,\mathrm{d}m = F\,\mathrm{d}t. \tag{1.4}$$

The term $\mathrm{d}m\,\mathrm{d}v$ is a product of two small quantities, so we drop it. Dividing by $\mathrm{d}t$,

$$m\frac{\mathrm{d}v}{\mathrm{d}t} = \underbrace{-v_e\frac{\mathrm{d}m}{\mathrm{d}t}}_{\text{thrust } T} + F. \tag{1.5}$$

Since $\mathrm{d}m/\,\mathrm{d}t = \dot{m} < 0$, the thrust $T = -\dot{m}\,v_e = |\dot{m}|\,v_e$ is positive. Engineers usually quote the *specific impulse* I_{sp} instead of v_e, with $v_e = I_{sp}\,g_0$ and $g_0 = 9.806\,65\,\mathrm{m/s^2}$.

Now switch off external forces ($F = 0$) and separate variables:

$$\mathrm{d}v = -v_e\frac{\mathrm{d}m}{m} \quad\Longrightarrow\quad \int_{v_0}^{v_f}\mathrm{d}v = -v_e\int_{m_0}^{m_f}\frac{\mathrm{d}m}{m} \quad\Longrightarrow\quad \boxed{\Delta v = v_e\ln\frac{m_0}{m_f}} \tag{1.6}$$

This is the *Tsiolkovsky rocket equation*. If the rocket instead climbs straight up against constant gravity g for a burn time t_b, then $F = -mg$ and the same steps give

$$\Delta v = v_e\ln\frac{m_0}{m_f} - g\,t_b, \tag{1.7}$$

where $g\,t_b$ is the *gravity loss*: velocity spent just holding the rocket up.

> **Worked example**: Why rockets have stages. Take a kerosene–oxygen engine with $I_{sp} = 300\,\mathrm{s}$, so $v_e = 300 \times 9.80665 \approx 2942\,\mathrm{m/s}$. Suppose 90% of the lift-off mass is propellant, so the mass ratio is $m_0/m_f = 10$. Then
>
> $$\Delta v = 2942 \times \ln 10 \approx 2942 \times 2.3026 \approx 6774\,\mathrm{m/s}.$$
>
> Reaching low Earth orbit needs roughly 9.3 to 9.5 km/s once gravity and drag losses are included. A single stage with a mass ratio of 10 falls well short. Dropping empty tanks part way up (staging) resets the mass ratio and is the classical way around this. This is why the three-stage vehicles in Zipfel's ROCKET3 and HYPER simulations look the way they do.

In three dimensions, (1.5) becomes

$$m(t)\,\dot{\boldsymbol{v}} = \boldsymbol{T} + \boldsymbol{F}_{\text{aero}} + m(t)\,\boldsymbol{g}(\boldsymbol{r}), \qquad \dot{m} = -\frac{\|\boldsymbol{T}\|}{I_{sp}\,g_0}, \tag{1.8}$$

with $\dot{\boldsymbol{r}} = \boldsymbol{v}$. These equations hold in an inertial frame. Writing them in Earth-fixed or body coordinates brings in extra Coriolis and centripetal terms; Chapter 2 and Chapter 3 handle that properly.

1.2.3 Rotation with changing inertia

For rotation, the analogue of Newton's law is *Euler's law*: the rate of change of angular momentum $\boldsymbol{H}$ about the centre of mass equals the applied moment $\boldsymbol{M}$. For a rigid body

$\boldsymbol{H} = \mathbf{J}\boldsymbol{\omega}$, where $\mathbf{J}$ is the 3×3 inertia tensor. When we write this in the body frame (which itself rotates with $\boldsymbol{\omega}$) and allow $\mathbf{J}$ to change as propellant drains, the product rule gives

$$\mathbf{J}\,\dot{\boldsymbol{\omega}} + \dot{\mathbf{J}}\,\boldsymbol{\omega} + \boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega}) = \boldsymbol{M}_{\text{aero}} + \boldsymbol{M}_{\text{thrust}}. \tag{1.9}$$

The term $\boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega})$ appears because the body frame rotates; the term $\dot{\mathbf{J}}\boldsymbol{\omega}$ appears because mass is leaving. (A full treatment also includes a small "jet damping" moment from the exhaust carrying angular momentum away. We derive it in Chapter 3; here we only want the shape of the equation.)

The thrust moment is where control enters. If the engine sits a distance ℓ behind the centre of mass and is gimballed by a small angle δ, the side force is about $T \sin\delta \approx T\delta$ and the pitching moment is

$$M_{\text{thrust}} \approx -\,T\,\ell\,\delta. \tag{1.10}$$

The sign depends on how we define δ; what matters is that the moment is proportional to the deflection. We return to this in Section 1.3.

1.2.4 Attitude with quaternions

Attitude can be described with three Euler angles, but those equations become singular when the pitch angle reaches $\pm 90^\circ$ — exactly what happens to a rocket standing on the pad. We therefore use a unit quaternion $\boldsymbol{q} = (q_0, q_1, q_2, q_3)$ with $q_0^2 + q_1^2 + q_2^2 + q_3^2 = 1$. Its kinematics are linear in $\boldsymbol{q}$:

$$\dot{\boldsymbol{q}} = \tfrac{1}{2}\,\Omega(\boldsymbol{\omega})\,\boldsymbol{q}, \qquad \Omega(\boldsymbol{\omega}) = \begin{bmatrix} 0 & -\omega_1 & -\omega_2 & -\omega_3 \\ \omega_1 & 0 & \omega_3 & -\omega_2 \\ \omega_2 & -\omega_3 & 0 & \omega_1 \\ \omega_3 & \omega_2 & -\omega_1 & 0 \end{bmatrix}. \tag{1.11}$$

Proposition 1.1. *The quaternion equation* (1.11) *keeps* $\|\boldsymbol{q}\|$ *constant.*

Proof. Ω is skew-symmetric, $\Omega^{\mathsf{T}} = -\Omega$, so for any vector $\boldsymbol{q}$ the scalar $\boldsymbol{q}^{\mathsf{T}}\Omega\boldsymbol{q}$ equals its own negative and must be zero. Then

$$\frac{\mathrm{d}}{\mathrm{d}t}\,\|\boldsymbol{q}\|^2 = 2\,\boldsymbol{q}^{\mathsf{T}}\dot{\boldsymbol{q}} = \boldsymbol{q}^{\mathsf{T}}\Omega(\boldsymbol{\omega})\,\boldsymbol{q} = 0. \qquad \square$$

> **Reality check.** The proof holds for the exact solution. A numerical integrator makes small errors at every step, so $\|\boldsymbol{q}\|$ slowly drifts away from 1. Real simulators re-normalise the quaternion, $\boldsymbol{q} \leftarrow \boldsymbol{q}/\,\|\boldsymbol{q}\|$, after every step.

1.2.5 Solving the equations on a computer

We cannot solve (1.2) with pen and paper for a real rocket, so we step forward in time with a step size h. The simplest method is *forward Euler*:

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + h\,\boldsymbol{f}(\boldsymbol{x}_k, \boldsymbol{u}_k, t_k). \tag{1.12}$$

Its error per step is $O(h^2)$ and its error at a fixed final time is $O(h)$: halving the step halves the error. The classical fourth-order Runge–Kutta method (RK4) evaluates $\boldsymbol{f}$ four times per step,

$$\begin{aligned} \boldsymbol{k}_1 &= \boldsymbol{f}(\boldsymbol{x}_k, t_k), & \boldsymbol{k}_2 &= \boldsymbol{f}\left(\boldsymbol{x}_k + \tfrac{h}{2}\boldsymbol{k}_1,\ t_k + \tfrac{h}{2}\right), \\ \boldsymbol{k}_3 &= \boldsymbol{f}\left(\boldsymbol{x}_k + \tfrac{h}{2}\boldsymbol{k}_2,\ t_k + \tfrac{h}{2}\right), & \boldsymbol{k}_4 &= \boldsymbol{f}(\boldsymbol{x}_k + h\boldsymbol{k}_3,\ t_k + h), \\ \boldsymbol{x}_{k+1} &= \boldsymbol{x}_k + \tfrac{h}{6}\left(\boldsymbol{k}_1 + 2\boldsymbol{k}_2 + 2\boldsymbol{k}_3 + \boldsymbol{k}_4\right), \end{aligned} \tag{1.13}$$

and its error at a fixed final time is $O(h^4)$: halving the step cuts the error by a factor of about 16.

Accuracy is not the only concern. A step that is too large can make the numerical solution blow up even when the real system is perfectly calm.

> **Worked example**: Choosing a step size for a fast actuator. Model a gimbal actuator as a first-order lag, $\dot{\delta} = -\lambda(\delta - \delta_{\text{cmd}})$, with time constant $1/\lambda = 20\,\text{ms}$, so $\lambda = 50/\text{s}$. Look at the error $e = \delta - \delta_{\text{cmd}}$ with a constant command: $\dot{e} = -\lambda e$. Forward Euler gives
>
> $$e_{k+1} = e_k - h\lambda e_k = (1 - h\lambda)\, e_k.$$
>
> The numerical error shrinks only if $|1 - h\lambda| < 1$, which means $0 < h < 2/\lambda$. With $\lambda = 50$ this is $h < 0.04\,\text{s}$. At $h = 0.05\,\text{s}$ the factor is $1 - 2.5 = -1.5$, and the simulated actuator oscillates with growing amplitude even though the real one settles smoothly. The fastest part of the model sets the step size. This is why 6-DOF simulators typically run at 1 kHz or faster.

1.2.6 Monte Carlo: simulating thousands of flights

A single simulation answers "what happens on a perfect day?" But winds, engine thrust, sensor biases and aerodynamic coefficients are never exactly as modelled. We treat each uncertain parameter as a random variable, draw a sample, run the simulation, and repeat N times. From the N outcomes $y_1, \dots, y_N$ (for example, landing miss distance) we estimate the mean and spread:

$$\bar{y} = \frac{1}{N}\sum_{i=1}^{N} y_i, \qquad s^2 = \frac{1}{N-1}\sum_{i=1}^{N} (y_i - \bar{y})^2. \tag{1.14}$$

The standard error of $\bar{y}$ is $s/\sqrt{N}$, so to halve the uncertainty in the mean we need four times as many runs. If we want the probability p of a rare event, such as missing the landing pad, the relative error of the estimate $\hat{p}$ is about

$$\frac{\sqrt{p(1-p)/N}}{p} \approx \frac{1}{\sqrt{Np}} \quad (p \ll 1). \tag{1.15}$$

To estimate a 1-in-1000 failure rate to within about 10%, we need $Np \approx 100$, so $N \approx 100\,000$ runs. Rare failures are expensive to measure by simulation alone, a point we return to in Chapter 12 and Chapter 16.

1.3 Feedback: correcting what we did not predict

Think about steering a car. You do not work out every turn of the steering wheel before you leave home and then drive with your eyes shut. You look at the road, notice that you are drifting towards the edge of your lane, and turn the wheel a little to correct it. Then you look again. That loop — *look, compare, correct, repeat* — is **feedback**.

A rocket can be flown in two ways:

- **Open-loop control.** Compute the whole gimbal history in advance from the model and play it back, no matter what actually happens. This works only if the model is perfect and nothing unexpected happens. Neither is ever true.
- **Closed-loop (feedback) control.** Measure what the vehicle is actually doing, compare it with what we wanted, and use the difference to decide the next command.

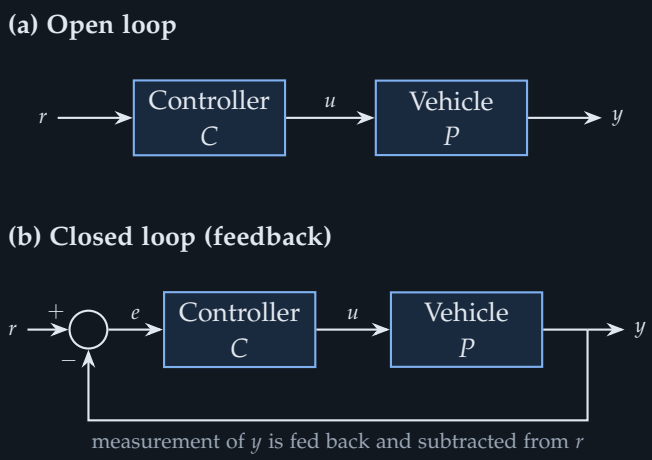


Figure 1.2: (a) Open-loop control only looks forward. (b) Feedback measures the output y, subtracts it from the desired value r to get the error e, and lets the controller act on that error. The circle is a *summing junction*: it adds the signals coming in, with the signs shown.

Åström and Murray name two great powers of feedback: it makes a system *robust to uncertainty* (it works even when our model is wrong), and it can *change the dynamics of a system* (it can make an unstable vehicle stable). This section shows both with mathematics. First, though, we need a few basic tools. If you have already met transfer functions, skim Sections 1.3.1 to 1.3.3 and continue from Section 1.3.4.

1.3.1 The language of control: signals, systems and block diagrams

Control engineers describe everything with two ideas.

A signal is a quantity that changes with time: the pitch angle $\theta(t)$, the gimbal angle $\delta(t)$, the wind speed, a sensor reading.

A system is anything that takes an input signal and produces an output signal. We draw it as a box with an arrow in and an arrow out. The gimbal actuator is a system: its input is the commanded angle, its output is the actual angle.

We will use the same letters for the same roles throughout the book:

Symbol	Name	On our launch vehicle
r	reference (what we want)	the pitch angle guidance asks for
y	output (what we get)	the actual pitch angle
$e = r - y$	error	how far off we are
u	control input	the gimbal command sent to the actuator
d	disturbance	a wind gust pushing the nose sideways
n	sensor noise	small random errors in the gyro reading
P	plant	the thing being controlled: actuator plus vehicle
C	controller	the flight software that turns e into u

The word *plant* is old engineering language from the days of controlling chemical and power plants. It just means "the system we are trying to control".

1.3.2 What is a transfer function?

The problem with differential equations. Systems are described by differential equations. Here is a simple, realistic one. A gimbal actuator does not jump instantly to the commanded angle δ_{cmd}; it moves towards it at a speed proportional to how far away it still is:

$$\tau\,\dot{\delta}(t) + \delta(t) = \delta_{\text{cmd}}(t), \tag{1.16}$$

where τ is the actuator's *time constant* (a typical value is $\tau = 0.02\,\text{s}$). That is fine on its own. But in a real loop the actuator feeds the vehicle, the vehicle feeds the sensor, the sensor feeds the filter, the filter feeds the controller, and the controller feeds the actuator. Chaining five differential equations together by hand is slow and error-prone. We want a way to treat each box as something we can simply *multiply*.

The Laplace transform. The tool that does this is the *Laplace transform*. It turns a function of time $f(t)$ into a function of a new variable s:

$$F(s) = \mathcal{L}\{f(t)\} = \int_0^\infty f(t)\,e^{-st}\,\mathrm{d}t. \tag{1.17}$$

You can think of s as a "rate" variable: the transform measures how much of each exponential e^{st} is inside the signal. You do not need to compute these integrals often. What matters is one property.

Proposition 1.2 (Differentiation becomes multiplication by s)**.**

$$\mathcal{L}\{\dot{f}(t)\} = s\,F(s) - f(0). \tag{1.18}$$

Proof. Integrate by parts, with $\int u\,\mathrm{d}v = uv - \int v\,\mathrm{d}u$, taking $u = e^{-st}$ and $\mathrm{d}v = \dot{f}\,\mathrm{d}t$:

$$\int_0^\infty \dot{f}(t)\,e^{-st}\,\mathrm{d}t = \Big[f(t)\,e^{-st}\Big]_0^\infty - \int_0^\infty f(t)\,(-s)\,e^{-st}\,\mathrm{d}t = \big(0 - f(0)\big) + s\,F(s).$$

(The term at $t \to \infty$ vanishes for the signals we care about, because e^{-st} shrinks faster than f grows.) □

If the system starts at rest, $f(0) = 0$, and the rule is simply: *taking a derivative in time is the same as multiplying by s*. Applying it twice gives $\mathcal{L}\{\ddot{f}\} = s^2F(s)$ when the system starts at rest. Calculus has become algebra.

Three transforms are worth remembering:

Time signal $f(t)$	**Laplace transform $F(s)$**	**Meaning**
1 for $t \geq 0$ (a step)	$\dfrac{1}{s}$	a command that switches on and stays on
e^{-at}	$\dfrac{1}{s+a}$	an exponential decay with rate a
$\dot{f}(t)$	$sF(s) - f(0)$	a derivative

(For the second one: $\int_0^\infty e^{-at}e^{-st}\,\mathrm{d}t = \int_0^\infty e^{-(s+a)t}\,\mathrm{d}t = \frac{1}{s+a}$.)

From differential equation to transfer function. Now transform the actuator equation (1.16), starting from rest. Write $\Delta(s)$ for the transform of $\delta(t)$ and $\Delta_{\text{cmd}}(s)$ for the transform of $\delta_{\text{cmd}}(t)$:

$$\tau\, s\, \Delta(s) + \Delta(s) = \Delta_{\text{cmd}}(s) \quad \Longrightarrow \quad (\tau s + 1)\, \Delta(s) = \Delta_{\text{cmd}}(s) \quad \Longrightarrow \quad \frac{\Delta(s)}{\Delta_{\text{cmd}}(s)} = \frac{1}{\tau s + 1}. \tag{1.19}$$

That ratio is the actuator's transfer function.

Definition 1.3. The *transfer function* of a system is the ratio of the Laplace transform of its output to the Laplace transform of its input, with the system starting at rest:

$$G(s) = \frac{Y(s)}{U(s)} \qquad \Longleftrightarrow \qquad Y(s) = G(s)\, U(s). \tag{1.20}$$

> **Key idea.** A transfer function is the whole personality of a system packed into one formula. Whatever input you give it, the output is just "transfer function times input" — one multiplication. That is what makes long chains of systems easy to handle.

What a transfer function tells you. We can read three useful things straight off $G(s) = 1/(\tau s + 1)$ without solving any equation.

1. The steady-state gain. Setting $s = 0$ means "no change over time". Here $G(0) = 1$: if you hold a constant command, the actuator eventually reaches exactly that angle. In general, $G(0)$ is the ratio of output to input once everything has settled.

2. The speed of response, from the poles. The *poles* of a transfer function are the values of s that make its denominator zero. Here $\tau s + 1 = 0$ gives one pole at $s = -1/\tau$. Each pole p produces a term e^{pt} in the response. For a step command the actuator angle is

$$\delta(t) = \delta_{\text{cmd}} \left(1 - e^{-t/\tau}\right), \tag{1.21}$$

which reaches 63% of the command after one time constant ($1 - e^{-1} \approx 0.632$) and 98% after four ($1 - e^{-4} \approx 0.982$). With $\tau = 0.02\,\text{s}$ the pole is at $s = -50$ and the actuator is essentially there after 80 ms.

The sign of the pole decides everything:

Pole p	**Term in response**	**Behaviour**
negative real, e.g. -50	e^{-50t}	dies away quickly: *stable*
positive real, e.g. $+2$	e^{2t}	grows without limit: *unstable*
complex, $\sigma \pm i\omega$	$e^{\sigma t}\cos(\omega t)$	oscillates; stable only if $\sigma < 0$

Rule: a system is stable when every pole has a negative real part. Keep this rule in mind; we use it below to show that feedback can stabilise a rocket.

3. How it handles fast and slow commands. If the input is a sine wave of angular frequency ω (in rad/s), the output of a stable system is a sine wave of the same frequency, scaled by $|G(i\omega)|$ and delayed by the angle of $G(i\omega)$. For the actuator, $G(i\omega) = 1/(1 + i\omega\tau)$, so

$$|G(i\omega)| = \frac{1}{\sqrt{1 + (\omega\tau)^2}}, \qquad \angle G(i\omega) = -\arctan(\omega\tau). \tag{1.22}$$

With $\tau = 0.02\,\text{s}$:

ω (rad/s)	$\lvert G(i\omega)\rvert$	phase	**What it means**
10	0.98	$-11°$	slow commands are followed almost perfectly
50	0.71	$-45°$	at $\omega = 1/\tau$ the actuator starts to fall behind
200	0.24	$-76°$	fast commands are mostly ignored, and late

So the actuator behaves like a *low-pass filter*: it passes slow signals and blocks fast ones. This frequency view is how flight-control engineers judge stability margins in Chapter 11.

A second example: the rocket's pitch motion. We now take the pitch equation from Section 1.3.6 below, $\ddot{\theta} = \mu_\alpha \theta + \mu_c \delta$, and transform it from rest:

$$s^2\,\Theta(s) = \mu_\alpha\,\Theta(s) + \mu_c\,\Delta(s) \quad \Longrightarrow \quad P_\theta(s) = \frac{\Theta(s)}{\Delta(s)} = \frac{\mu_c}{s^2 - \mu_\alpha}. \tag{1.23}$$

Its poles solve $s^2 = \mu_\alpha$, so $s = \pm\sqrt{\mu_\alpha}$. One pole is positive: the uncontrolled rocket is unstable. We will come back to this.

> **Reality check**: Linear, time-invariant only. Transfer functions only work for systems that are *linear* (double the input, double the output) and *time-invariant* (the same today as tomorrow). A rocket is neither: its mass, speed and air density change all the time. Engineers handle this by freezing the vehicle at one moment of flight, approximating it as linear around that point, designing a controller there, and repeating for many points along the trajectory. Blending those controllers is *gain scheduling* (Chapter 11).

1.3.3 Block diagram algebra

Because each box is "output = transfer function $\times$ input", connecting boxes becomes simple algebra. There are three patterns.

Boxes in series multiply. If $X = G_1 U$ and $Y = G_2 X$, then

$$Y = G_2 (G_1 U) = (G_2 G_1) U. \tag{1.24}$$

For example, the actuator $1/(\tau s + 1)$ feeding the pitch dynamics (1.23) gives the plant

$$P(s) = \frac{\mu_c}{s^2 - \mu_\alpha} \cdot \frac{1}{\tau s + 1}. \tag{1.25}$$

Boxes in parallel add. If two boxes receive the same input and their outputs are added, $Y = G_1 U + G_2 U = (G_1 + G_2) U$.

The feedback loop. This is the important one. Take the closed loop in Fig. 1.2(b) and write down what each part does:

$$E = R - Y \qquad \text{(the summing junction: error = wanted } - \text{ actual)} \tag{1.26}$$

$$U = C E \qquad \text{(the controller acts on the error)} \tag{1.27}$$

$$Y = P U \qquad \text{(the plant responds to the command)} \tag{1.28}$$

Substitute the first two into the third, one step at a time:

$$Y = P C E = PC (R - Y) \tag{1.29}$$

$$Y = PC R - PC Y \tag{1.30}$$

$$Y + PC Y = PC R \tag{1.31}$$

$$(1 + PC) Y = PC R \tag{1.32}$$

$$Y = \frac{PC}{1 + PC} R. \tag{1.33}$$

The product $L = PC$ is called the *loop gain*: the total gain all the way around the loop. We give the closed-loop transfer function a name:

$$T = \frac{PC}{1 + PC} = \frac{L}{1 + L}. \tag{1.34}$$

> **Key idea**: Why high loop gain makes the output follow the command. Look at $T = L/(1 + L)$ when L is large. With $L = 10$, $T = 10/11 \approx 0.91$. With $L = 100$, $T = 100/101 \approx 0.99$. With $L = 1000$, $T \approx 0.999$. As the loop gain grows, T gets closer and closer to 1, and so $Y \approx R$ *whatever P is*. The plant almost drops out of the answer. That is the secret of feedback.

1.3.4 Feedback reduces sensitivity to model error

Our model of the plant is never exact. Thrust varies from engine to engine, aerodynamic coefficients come from wind tunnels with error bars, and the mass changes with every second of burn. So the real question is: *if P is wrong, how wrong is the result?*

First, a numerical experiment. Take a plant whose gain we believe is $P = 10$, but which is really 20% higher, $P = 12$.

- **Open loop.** To get $y = r$ we choose $C = 1/10$, so we expect $y = PC\,r = r$. If the real plant is $P = 12$, we get $y = 1.2\,r$ — a 20% error. Every bit of model error goes straight into the output.
- **Closed loop with** $C = 1$ (so $L = PC = 10$). With the model, $T = 10/11 = 0.9091$. With the real plant, $T = 12/13 = 0.9231$. The output changed by only $(0.9231 - 0.9091)/0.9091 \approx 1.5\%$.
- **Closed loop with** $C = 10$ (so $L = 100$). With the model, $T = 100/101 = 0.9901$. With the real plant, $T = 120/121 = 0.9917$. The change is now only about 0.17%.

A 20% error in the plant became a 1.5% error, and then a 0.17% error, simply by closing the loop and raising the gain.

Now the general rule. We want to know how a small relative change in P turns into a relative change in T. Start by differentiating $T = PC/(1 + PC)$ with respect to P using the quotient rule $\left(\frac{f}{g}\right)' = \frac{f'g - fg'}{g^2}$, with $f = PC$, $f' = C$, $g = 1 + PC$, $g' = C$:

$$\frac{\mathrm{d}T}{\mathrm{d}P} = \frac{C\,(1 + PC) - PC \cdot C}{(1 + PC)^2} = \frac{C + PC^2 - PC^2}{(1 + PC)^2} = \frac{C}{(1 + PC)^2}. \tag{1.35}$$

A relative change is a change divided by the value, so the ratio we want is $\dfrac{\mathrm{d}T/T}{\mathrm{d}P/P} = \dfrac{\mathrm{d}T}{\mathrm{d}P} \cdot \dfrac{P}{T}$. Since $\dfrac{P}{T} = P \cdot \dfrac{1 + PC}{PC} = \dfrac{1 + PC}{C}$,

$$\boxed{\frac{\mathrm{d}T/T}{\mathrm{d}P/P} = \frac{C}{(1 + PC)^2} \cdot \frac{1 + PC}{C} = \frac{1}{1 + PC} \;=:\; S.} \tag{1.36}$$

S is called the *sensitivity function*. It is the factor by which feedback shrinks the effect of model error:

$$\text{relative error in output} \;\approx\; S \times \text{relative error in plant model.} \tag{1.37}$$

For open loop there is no feedback and the factor is 1. With $L = 10$, $S = 1/11$, so a 20% model error gives about $20\%/11 \approx 1.8\%$. That matches our experiment (1.5%); the small difference is because a 20% change is not really "small", and (1.36) is exact only for small changes.

1.3.5 Disturbances, sensor noise and the trade-off $S + T = 1$

Real loops also have things pushing on them from outside. Add a disturbance d (a wind gust that moves the output directly) and sensor noise n (the gyro reports $y + n$ instead of y). The

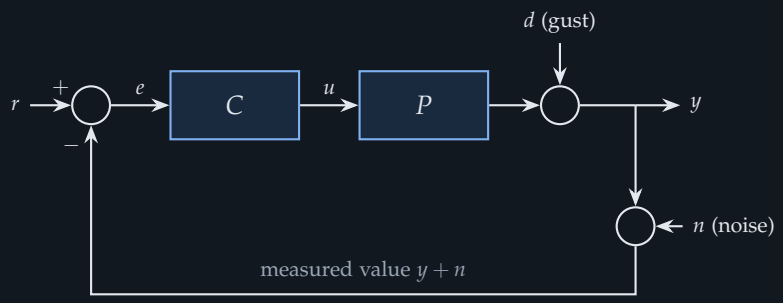


Figure 1.3: A feedback loop with a disturbance d acting on the output and noise n added by the sensor.

full loop is drawn in Fig. 1.3.

Write the loop equations again. The controller now sees the *measured* output $Y + N$:

$$E = R - (Y + N), \tag{1.38}$$

$$Y = P\,C\,E + D. \tag{1.39}$$

Substitute and collect the Y terms, exactly as before:

$$Y = PC\,(R - Y - N) + D \tag{1.40}$$

$$(1 + PC)\,Y = PC\,R - PC\,N + D \tag{1.41}$$

$$Y = \underbrace{\frac{PC}{1+PC}}_{T}\,R \;+\; \underbrace{\frac{1}{1+PC}}_{S}\,D \;-\; \underbrace{\frac{PC}{1+PC}}_{T}\,N. \tag{1.42}$$

This one line tells us almost everything about a feedback loop:

- the command R reaches the output through T: we want $T \approx 1$;
- the gust D reaches the output through S: we want $S \approx 0$;
- the sensor noise N reaches the output through T: we want $T \approx 0$.

And yet S and T are tied together. Add them:

$$S + T = \frac{1}{1+PC} + \frac{PC}{1+PC} = \frac{1+PC}{1+PC} = 1. \tag{1.43}$$

So we cannot make both small at once. If S is tiny (good gust rejection), T is close to 1 and the sensor noise passes straight through.

The way out is that S and T depend on frequency. Wind gusts and model errors are mostly *slow*; gyro noise is mostly *fast*. So a good flight controller has a large loop gain $|L(i\omega)|$ at low frequencies (making S small there) and a small loop gain at high frequencies (making T small there). Shaping L this way is called *loop shaping*, and much of Chapter 11 is about doing it well.

1.3.6 Feedback can stabilise an unstable vehicle

Why a rocket is unstable. Think of a dart or an arrow. Its feathers put the *centre of pressure* (the point where the air's push effectively acts) *behind* the *centre of gravity*. If the nose tips to

one side, the air pushes the tail back into line: the arrow is stable. A launch vehicle is usually the other way round. It has no large tail fins, and its long body and nose put the centre of pressure *ahead* of the centre of gravity. Now, when the nose tips a little, the air pushes it further. It is like trying to throw an arrow backwards.

Near one flight condition the pitch angle θ approximately obeys

$$\ddot{\theta} = \mu_\alpha \, \theta + \mu_c \, \delta, \qquad \mu_\alpha > 0, \ \mu_c > 0, \tag{1.44}$$

where μ_α measures how strongly the air pushes the nose away (the instability), and μ_c measures how strongly the gimbal angle δ turns the vehicle (the control effectiveness). We choose the sign of δ so that $\mu_c > 0$.

Without control. Set $\delta = 0$ and try a solution $\theta = e^{st}$. Then $\ddot{\theta} = s^2 e^{st}$, so $s^2 = \mu_\alpha$ and $s = \pm\sqrt{\mu_\alpha}$. These are exactly the poles of (1.23). One is positive. With $\mu_\alpha = 4/\mathrm{s}^2$ the poles are ± 2, so a small error grows like e^{2t}: it doubles every $\ln 2/2 \approx 0.35\,\mathrm{s}$, and after 2 s it is $e^4 \approx 55$ times larger. Without active control, the vehicle would tumble within seconds.

With proportional–derivative (PD) feedback. We want to keep θ at zero, so the error is $e = 0 - \theta$. Let the controller react to the error and to how fast it is changing:

$$\delta = -k_p \, \theta - k_d \, \dot{\theta}. \tag{1.45}$$

Here k_p (the proportional gain) pushes back harder the further the nose has turned, and k_d (the derivative gain) pushes back against the turning speed, like a shock absorber. Substituting into (1.44):

$$\ddot{\theta} = \mu_\alpha \theta + \mu_c \left(-k_p \theta - k_d \dot{\theta} \right) \tag{1.46}$$

$$\ddot{\theta} + \mu_c k_d \, \dot{\theta} + (\mu_c k_p - \mu_\alpha) \, \theta = 0. \tag{1.47}$$

In transfer-function language, the controller is $C(s) = k_p + k_d s$ and the closed-loop poles are the roots of $1 + P_\theta(s) C(s) = 0$:

$$1 + \frac{\mu_c \, (k_p + k_d s)}{s^2 - \mu_\alpha} = 0 \quad \Longrightarrow \quad s^2 + \mu_c k_d \, s + (\mu_c k_p - \mu_\alpha) = 0, \tag{1.48}$$

the same equation, reached the other way.

For a quadratic $s^2 + a_1 s + a_0 = 0$, both roots have negative real parts exactly when $a_1 > 0$ and $a_0 > 0$. (Their sum is $-a_1$ and their product is a_0; two roots with negative real parts must have a negative sum and a positive product.) So the closed loop is stable if and only if

$$k_d > 0 \qquad \text{and} \qquad k_p > \frac{\mu_\alpha}{\mu_c}. \tag{1.49}$$

In words: the proportional gain must be large enough to overpower the air's destabilising push, and the derivative gain must add some damping.

Worked example: Designing a pitch controller. Let $\mu_\alpha = 4/\text{s}^2$ and $\mu_c = 8/\text{s}^2$ per radian. We want the closed loop to behave like a standard second-order system $\ddot{\theta} + 2\zeta\omega_n\dot{\theta} + \omega_n^2\theta = 0$, with natural frequency $\omega_n = 3\,\text{rad/s}$ (how fast it responds) and damping ratio $\zeta = 0.7$ (how little it overshoots; 0.7 is a common, well-behaved choice). Match the coefficients of (1.47):

$$\mu_c k_p - \mu_\alpha = \omega_n^2 \;\Rightarrow\; k_p = \frac{9+4}{8} = 1.625, \qquad \mu_c k_d = 2\zeta\omega_n \;\Rightarrow\; k_d = \frac{4.2}{8} = 0.525\,\text{s}.$$

Check against (1.49): $k_p = 1.625 > 4/8 = 0.5$ and $k_d > 0$, so the loop is stable. Now check the hardware: a sudden pitch error of $2^\circ = 0.0349\,\text{rad}$ asks for $\delta = 1.625 \times 0.0349 \approx 0.057\,\text{rad} \approx 3.3^\circ$. Typical gimbal limits are around $\pm 5^\circ$ to $\pm 8^\circ$, so this design is feasible — but a 4° error would already saturate a $\pm 5^\circ$ gimbal. Limits like this are why we must model actuators carefully (Chapter 5).

Key idea. Feedback did not just "correct errors". It moved the poles from ± 2 (one unstable) to $-\zeta\omega_n \pm i\,\omega_n\sqrt{1-\zeta^2} = -2.1 \pm 2.14i$ (both stable). Placing the poles where we want them is the heart of control design, and LQR (Chapter 11) is a systematic way to do it.

1.3.7 Steady-state error and integral action

There is one more thing to fix. We saw that with $L = 10$, $T = 10/11$: the output settles at 91% of the command, not 100%. Here is why, and how to cure it.

Take a simple first-order plant $\dot{y} = -a\,y + b\,u$, so $P(s) = \dfrac{b}{s+a}$, and a proportional controller $u = k\,(r - y)$, so $C(s) = k$. The closed-loop equation is

$$\dot{y} = -(a + bk)\,y + bk\,r. \tag{1.50}$$

When things settle, $\dot{y} = 0$, so

$$y_{ss} = \frac{bk}{a+bk}\,r, \qquad e_{ss} = r - y_{ss} = \frac{a}{a+bk}\,r. \tag{1.51}$$

You can get the same answer from transfer functions: setting $s = 0$ gives $T(0) = \dfrac{bk/a}{1 + bk/a} = \dfrac{bk}{a+bk}$. The error gets smaller as k grows, but it never reaches zero, and very high gains cause other problems (actuator saturation, noise).

The fix is to add an *integral* term, which adds up the error over time:

$$u(t) = k_p\,e(t) + k_i \int_0^t e(\tau)\;\mathrm{d}\tau, \qquad C(s) = k_p + \frac{k_i}{s}. \tag{1.52}$$

(Integration is the opposite of differentiation, so in the Laplace world it is *division* by s.) There are two ways to see why this removes the error:

- **In time.** When everything has settled, the integral must stop changing. The only way it stops changing is if the thing being integrated, e, is zero.

- **In transfer functions.** As $s \to 0$, the term k_i/s becomes infinitely large, so the loop gain $L(0)$ is infinite. Then $T(0) = L/(1+L) \to 1$ and $S(0) \to 0$: perfect tracking of constant commands and perfect rejection of constant disturbances, such as a steady crosswind.

Proportional, integral and derivative together give the **PID controller**, the workhorse of practical control that we study in Chapter 11.

> **Key idea**: What to remember from this section.
>
> - A *transfer function* $G(s) = Y(s)/U(s)$ turns a differential equation into algebra: a derivative becomes a multiplication by s.
> - *Poles* decide stability: every pole must have a negative real part.
> - Series boxes multiply; a feedback loop gives $T = L/(1+L)$ with loop gain $L = PC$.
> - Feedback shrinks model error by the sensitivity $S = 1/(1+L)$, and $S + T = 1$ forces a trade-off between rejecting gusts and passing sensor noise.
> - Feedback can stabilise an unstable rocket if $k_p > \mu_\alpha/\mu_c$ and $k_d > 0$.
> - Integral action removes steady-state error.

1.4 Estimation: living with noisy sensors

The controller in Section 1.3 used θ and $\dot{\theta}$ as if we knew them exactly. We do not. Gyroscopes drift, accelerometers have biases, GPS jumps by metres and drops out, barometers are affected by the weather. Navigation combines all of these, together with a model of how the vehicle moves, into one best estimate.

1.4.1 Predict, then correct: the g–h filter

Labbe's book starts with the simplest useful idea. Keep an estimate of position $\hat{x}$ and velocity $\hat{\dot{x}}$. At each time step:

$$\text{predict:} \qquad \hat{x}^- = \hat{x} + \hat{\dot{x}}\,\Delta t, \tag{1.53}$$

$$\text{residual:} \qquad \rho = z - \hat{x}^-, \tag{1.54}$$

$$\text{correct:} \qquad \hat{x} = \hat{x}^- + g\,\rho, \qquad \hat{\dot{x}} \leftarrow \hat{\dot{x}} + h\,\frac{\rho}{\Delta t}. \tag{1.55}$$

The prediction uses the model; the measurement z corrects it. The gains $g, h \in (0,1)$ decide how much we trust the measurement over the prediction. Large gains follow noisy data closely; small gains smooth heavily but react slowly. The Kalman filter is, at heart, a principled way to choose these gains at every step.

1.4.2 Combining two uncertain numbers

Here is the key calculation. Our prediction says the altitude is μ_p with uncertainty (variance) σ_p^2. A barometer reads z with variance σ_z^2. Model both as Gaussian. Bayes' rule says the updated belief is proportional to prior times likelihood:

$$p(x \mid z) \;\propto\; p(z \mid x)\, p(x) \;\propto\; \exp\!\left(-\frac{(x-\mu_p)^2}{2\sigma_p^2}\right) \exp\!\left(-\frac{(z-x)^2}{2\sigma_z^2}\right). \tag{1.56}$$

The exponent is a quadratic in x, so the result is again Gaussian. Collecting the x^2 and x terms and completing the square gives the posterior mean μ and variance σ^2:

$$\frac{1}{\sigma^2} = \frac{1}{\sigma_p^2} + \frac{1}{\sigma_z^2}, \qquad \mu = \sigma^2 \left(\frac{\mu_p}{\sigma_p^2} + \frac{z}{\sigma_z^2}\right). \tag{1.57}$$

Two things are worth noticing. First, *precisions add*: $1/\sigma^2$ is the sum of the two precisions, so the fused variance is smaller than either input variance. Second, the mean is a weighted average, with more weight on whichever source is more certain.

We can rewrite (1.57) in "predict plus correction" form. Define the *Kalman gain*

$$K = \frac{\sigma_p^2}{\sigma_p^2 + \sigma_z^2} \in (0, 1). \tag{1.58}$$

Then a little algebra turns (1.57) into

$$\boxed{\mu = \mu_p + K\,(z - \mu_p), \qquad \sigma^2 = (1 - K)\,\sigma_p^2.} \tag{1.59}$$

Compare with (1.55): K plays exactly the role of g, but now it is computed from the uncertainties rather than chosen by hand.

> **Worked example**: Fusing a prediction with a barometer. The prediction says altitude $\mu_p = 1200\,\text{m}$ with $\sigma_p = 10\,\text{m}$. The barometer reads $z = 1212\,\text{m}$ with $\sigma_z = 5\,\text{m}$. Then
>
> $$K = \frac{100}{100 + 25} = 0.8, \qquad \mu = 1200 + 0.8 \times 12 = 1209.6\,\text{m},$$
> $$\sigma^2 = 0.2 \times 100 = 20\,\text{m}^2, \qquad \sigma \approx 4.47\,\text{m}.$$
>
> The fused uncertainty, 4.47 m, is smaller than both the prediction (10 m) and the sensor (5 m). Combining information never makes us less certain, as long as the noise models are honest.

1.4.3 The steady-state filter

Between measurements, the vehicle moves and our uncertainty grows. In the simplest model the true value takes a random step of variance q each time step, so the predict step adds q to the variance. Writing $r = \sigma_z^2$ and P^- for the variance after prediction, one full cycle is

$$P_{k+1}^- = \underbrace{\frac{P_k^-\, r}{P_k^- + r}}_{\text{after update}} + q. \tag{1.60}$$

After many steps P^- settles to a fixed value. Setting $P^-_{k+1} = P^-_k = P^-$ and multiplying out gives the quadratic $(P^-)^2 - qP^- - qr = 0$, whose positive root is

$$P^- = \frac{q + \sqrt{q^2 + 4qr}}{2}, \qquad K_\infty = \frac{P^-}{P^- + r}. \tag{1.61}$$

With $q = 1$ and $r = 25$ (both in m^2): $P^- = (1 + \sqrt{101})/2 \approx 5.525$ and $K_\infty \approx 5.525/30.525 \approx 0.181$. You can check the fixed point: the update gives $5.525 \times 25/30.525 \approx 4.525$, and adding $q = 1$ returns 5.525.

1.4.4 Looking ahead: the multivariate Kalman filter

For the full 14-state vehicle the same two steps become matrix equations. With a linear model $\boldsymbol{x}_{k+1} = \mathbf{F}\boldsymbol{x}_k + \mathbf{B}\boldsymbol{u}_k + \boldsymbol{w}_k$, measurement $\boldsymbol{z}_k = \mathbf{H}\boldsymbol{x}_k + \boldsymbol{v}_k$, process noise covariance $\mathbf{Q}$ and measurement noise covariance $\mathbf{R}$:

$$\text{predict:} \quad \hat{\boldsymbol{x}}^- = \mathbf{F}\hat{\boldsymbol{x}} + \mathbf{B}\boldsymbol{u}, \qquad \mathbf{P}^- = \mathbf{F}\mathbf{P}\mathbf{F}^{\mathsf{T}} + \mathbf{Q}, \tag{1.62}$$

$$\text{update:} \quad \mathbf{K} = \mathbf{P}^-\mathbf{H}^{\mathsf{T}}\left(\mathbf{H}\mathbf{P}^-\mathbf{H}^{\mathsf{T}} + \mathbf{R}\right)^{-1}, \qquad \hat{\boldsymbol{x}} = \hat{\boldsymbol{x}}^- + \mathbf{K}\left(\boldsymbol{z} - \mathbf{H}\hat{\boldsymbol{x}}^-\right), \tag{1.63}$$

$$\mathbf{P} = (\mathbf{I} - \mathbf{K}\mathbf{H})\,\mathbf{P}^-. \tag{1.64}$$

If you set every matrix to a 1×1 number, with $\mathbf{F} = \mathbf{H} = 1$, you get back (1.59) exactly. Launch dynamics are nonlinear, so in Chapter 7 we will use the extended and unscented versions of this filter.

1.5 Optimization: choosing the best possible trajectory

Guidance has to pick a thrust and attitude history that reaches the target while respecting the physics and the hardware. There are infinitely many histories that almost work; we want the best one, for example the one that uses the least propellant. That is an optimization problem.

1.5.1 The standard form

Boyd and Vandenberghe write every optimization problem the same way:

$$\begin{aligned} &\text{minimize} && f_0(\boldsymbol{x}) \\ &\text{subject to} && f_i(\boldsymbol{x}) \le 0, \quad i = 1, \dots, m, \\ &&& h_j(\boldsymbol{x}) = 0, \quad j = 1, \dots, p. \end{aligned} \tag{1.65}$$

Here $\boldsymbol{x}$ is the decision vector (for guidance: the thrust at each time step), f_0 is the cost (propellant used), the f_i are inequality constraints (thrust limits, gimbal limits, no flying underground) and the h_j are equality constraints (the equations of motion, the target state).

1.5.2 Why convexity matters

Definition 1.4. A set $\mathcal{S}$ is *convex* if the straight line between any two of its points stays in the set: for all $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{S}$ and $\theta \in [0,1]$, $\theta \boldsymbol{x} + (1-\theta)\boldsymbol{y} \in \mathcal{S}$. A function f is *convex* if

$$f\big(\theta \boldsymbol{x} + (1-\theta)\boldsymbol{y}\big) \le \theta f(\boldsymbol{x}) + (1-\theta) f(\boldsymbol{y}). \tag{1.66}$$

Problem (1.65) is a *convex optimization problem* when $f_0, \dots, f_m$ are convex and the h_j are affine (linear plus a constant).

Theorem 1.5. *In a convex optimization problem, every local minimum is a global minimum.*

Proof. Suppose $\boldsymbol{x}$ is a local minimum but some feasible $\boldsymbol{y}$ has $f_0(\boldsymbol{y}) < f_0(\boldsymbol{x})$. The feasible set is convex, so every point $\boldsymbol{z}_\theta = (1-\theta)\boldsymbol{x} + \theta \boldsymbol{y}$ with $\theta \in (0,1]$ is feasible. By convexity of f_0,

$$f_0(\boldsymbol{z}_\theta) \le (1-\theta) f_0(\boldsymbol{x}) + \theta f_0(\boldsymbol{y}) < f_0(\boldsymbol{x}).$$

Taking θ very small puts $\boldsymbol{z}_\theta$ as close to $\boldsymbol{x}$ as we like, with a strictly lower cost. That contradicts $\boldsymbol{x}$ being a local minimum. □

This theorem is why convexity matters so much for flight software. A solver for a convex problem cannot get stuck in a bad local minimum, and modern interior-point methods solve such problems reliably in a predictable number of iterations. For an autonomous vehicle, *predictable* is just as important as *fast*.

1.5.3 The simplest case: least squares

Fitting a model to data is the most common optimization we will do (for example, fitting drag coefficient against Mach number from wind-tunnel points). The problem is

$$\text{minimize} \quad \|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2. \tag{1.67}$$

The cost is a convex quadratic. Its gradient is $2\mathbf{A}^{\mathsf{T}}(\mathbf{A}\boldsymbol{x} - \boldsymbol{b})$; setting it to zero gives the *normal equations* $\mathbf{A}^{\mathsf{T}}\mathbf{A}\,\boldsymbol{x} = \mathbf{A}^{\mathsf{T}}\boldsymbol{b}$, and, when $\mathbf{A}$ has independent columns,

$$\boldsymbol{x}^\star = (\mathbf{A}^{\mathsf{T}}\mathbf{A})^{-1}\mathbf{A}^{\mathsf{T}}\boldsymbol{b}. \tag{1.68}$$

This is the same formula as linear regression in statistics.

1.5.4 Hardware limits as cones

Now a constraint that is specific to rockets. The thrust vector $\boldsymbol{T} \in \mathbb{R}^3$ cannot point anywhere: the gimbal allows at most an angle $\delta_{\max}$ away from the vehicle axis $\hat{\boldsymbol{e}}$. Because $\hat{\boldsymbol{e}}^{\mathsf{T}}\boldsymbol{T} = \|\boldsymbol{T}\|\cos\delta$, the condition $\delta \le \delta_{\max}$ is

$$\|\boldsymbol{T}\|_2 \;\le\; \frac{\hat{\boldsymbol{e}}^{\mathsf{T}}\boldsymbol{T}}{\cos\delta_{\max}}. \tag{1.69}$$

The left side is a Euclidean norm and the right side is linear in $\boldsymbol{T}$. A constraint of the form $\|\mathbf{A}\boldsymbol{x} + \boldsymbol{b}\|_2 \le \boldsymbol{c}^{\mathsf{T}}\boldsymbol{x} + d$ is called a *second-order cone constraint*, and it is convex. The maximum-thrust limit $\|\boldsymbol{T}\|_2 \le T_{\max}$ is convex too.

> **Reality check**: The one constraint that is not convex. Most liquid engines cannot throttle below some minimum, so we also need $\|\boldsymbol{T}\|_2 \geq T_{\min} > 0$. This set is a ball with a hole in the middle — not convex. The line between thrust $+\boldsymbol{T}$ and $-\boldsymbol{T}$ passes through zero thrust, which the engine cannot produce. A celebrated result in powered-descent guidance, *lossless convexification* (Açıkmeşe and Ploen, 2007), introduces a slack variable Γ with $\|\boldsymbol{T}\| \leq \Gamma$ and $T_{\min} \leq \Gamma \leq T_{\max}$, and proves that under mild conditions the convex relaxed problem has the same optimal solution as the original. We study it in Chapter 9.

1.5.5 A first guidance problem

Put the pieces together for a simple landing problem. Treat the vehicle as a point mass with roughly constant mass, controlled acceleration $\boldsymbol{a}_k$ and gravity $\boldsymbol{g}$, sampled every Δt seconds over N steps:

$$\begin{aligned} \text{minimize} \quad & \textstyle\sum_{k=0}^{N-1} \|\boldsymbol{a}_k\|_2 \, \Delta t \\ \text{subject to} \quad & \boldsymbol{r}_{k+1} = \boldsymbol{r}_k + \Delta t\, \boldsymbol{v}_k + \tfrac{1}{2}\Delta t^2 (\boldsymbol{a}_k + \boldsymbol{g}), \\ & \boldsymbol{v}_{k+1} = \boldsymbol{v}_k + \Delta t\, (\boldsymbol{a}_k + \boldsymbol{g}), \\ & \|\boldsymbol{a}_k\|_2 \leq a_{\max}, \\ & \big\|(r_{k,x},\, r_{k,y})\big\|_2 \tan\gamma \leq r_{k,z} \quad \text{(glide-slope cone)}, \\ & \boldsymbol{r}_0, \boldsymbol{v}_0 \text{ given}, \qquad \boldsymbol{r}_N = \boldsymbol{0}, \quad \boldsymbol{v}_N = \boldsymbol{0}. \end{aligned} \tag{1.70}$$

The dynamics are linear equalities, the cost is a sum of norms, and every inequality is a second-order cone. So (1.70) is a *second-order cone program* (SOCP), which off-the-shelf solvers handle in milliseconds for N in the hundreds. The glide-slope cone keeps the vehicle above a cone with half-angle measured from the ground, so it cannot approach the pad too low.

1.5.6 Lower bounds from duality

One more idea we will need when comparing classical and quantum solvers. Form the *Lagrangian* of (1.65) with multipliers $\lambda_i \geq 0$ and ν_j:

$$L(\boldsymbol{x}, \boldsymbol{\lambda}, \boldsymbol{\nu}) = f_0(\boldsymbol{x}) + \sum_i \lambda_i f_i(\boldsymbol{x}) + \sum_j \nu_j h_j(\boldsymbol{x}), \qquad g(\boldsymbol{\lambda}, \boldsymbol{\nu}) = \inf_{\boldsymbol{x}} L(\boldsymbol{x}, \boldsymbol{\lambda}, \boldsymbol{\nu}). \tag{1.71}$$

For any feasible $\tilde{\boldsymbol{x}}$ we have $f_i(\tilde{\boldsymbol{x}}) \leq 0$ and $h_j(\tilde{\boldsymbol{x}}) = 0$, so $L(\tilde{\boldsymbol{x}}, \boldsymbol{\lambda}, \boldsymbol{\nu}) \leq f_0(\tilde{\boldsymbol{x}})$, and therefore

$$g(\boldsymbol{\lambda}, \boldsymbol{\nu}) \leq L(\tilde{\boldsymbol{x}}, \boldsymbol{\lambda}, \boldsymbol{\nu}) \leq f_0(\tilde{\boldsymbol{x}}). \tag{1.72}$$

This is *weak duality*: every dual value is a lower bound on the best possible cost. It holds even for non-convex problems. That makes it valuable later: if a quantum algorithm returns a solution with cost c and a classical dual bound gives g, then the gap $c - g$ tells us how far from optimal the quantum answer could possibly be.

1.6 Quantum computation: what it is, and what it is not

The abstract of this book promises quantum algorithms "where quantum computational advantage is demonstrable". To judge that honestly we need the basic mathematics of quantum computing. This section covers just enough to read Chapter 8 onward.

1.6.1 One qubit

A classical bit is 0 or 1. A *qubit* is a unit vector in $\mathbb{C}^2$:

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle\,, \qquad |0\rangle = \begin{bmatrix}1\\0\end{bmatrix},\ |1\rangle = \begin{bmatrix}0\\1\end{bmatrix}, \qquad |\alpha|^2 + |\beta|^2 = 1. \tag{1.73}$$

When we *measure* the qubit we get 0 with probability $|\alpha|^2$ and 1 with probability $|\beta|^2$ (the *Born rule*), and the state collapses to the outcome we saw. Ignoring an overall phase, which cannot be measured, every qubit state can be written

$$|\psi\rangle = \cos\tfrac{\vartheta}{2}\,|0\rangle + e^{i\varphi}\sin\tfrac{\vartheta}{2}\,|1\rangle\,, \tag{1.74}$$

a point on the *Bloch sphere* with polar angle ϑ and azimuth φ. Notice the parallel with attitude: a single qubit's state is a direction on a sphere, and single-qubit gates are rotations of that sphere.

1.6.2 Gates are unitary matrices

Operations on qubits are 2×2 *unitary* matrices, $U^\dagger U = I$, where $\dagger$ means conjugate transpose. Unitarity preserves length, so probabilities still add to one. Three gates we use constantly:

$$X = \begin{bmatrix}0&1\\1&0\end{bmatrix}, \qquad Z = \begin{bmatrix}1&0\\0&-1\end{bmatrix}, \qquad H = \frac{1}{\sqrt{2}}\begin{bmatrix}1&1\\1&-1\end{bmatrix}. \tag{1.75}$$

X flips $|0\rangle \leftrightarrow |1\rangle$. Z leaves $|0\rangle$ alone and flips the sign of $|1\rangle$. The Hadamard gate H creates equal superpositions:

$$H\,|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} =: |+\rangle\,, \qquad H\,|1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}} =: |-\rangle\,. \tag{1.76}$$

1.6.3 Many qubits: tensor products and exponential state spaces

Two qubits live in $\mathbb{C}^2 \otimes \mathbb{C}^2 = \mathbb{C}^4$, with basis $|00\rangle, |01\rangle, |10\rangle, |11\rangle$. In general n qubits need 2^n complex amplitudes:

$$|\Psi\rangle = \sum_{b\in\{0,1\}^n} c_b\,|b\rangle\,, \qquad \sum_b |c_b|^2 = 1. \tag{1.77}$$

This exponential growth is both the promise and the catch. Storing the state of 50 qubits on a classical computer takes $2^{50} \approx 1.13\times 10^{15}$ amplitudes; at 16 bytes per complex number that is about 1.8×10^{16} bytes, or 18 petabytes. But we can never *read out* those amplitudes

directly: a measurement returns just n classical bits. Quantum algorithms work by arranging interference so that the useful answers have large amplitude when we finally measure.

The two-qubit controlled-NOT gate flips the second qubit when the first is $|1\rangle$: $\mathrm{CNOT}\,|a\rangle\,|b\rangle = |a\rangle\,|a \oplus b\rangle$. Applying H to the first qubit and then CNOT to $|00\rangle$ gives

$$|00\rangle \xrightarrow{H\otimes I} \frac{|00\rangle + |10\rangle}{\sqrt{2}} \xrightarrow{\mathrm{CNOT}} \frac{|00\rangle + |11\rangle}{\sqrt{2}}, \tag{1.78}$$

a *Bell state* (Fig. 1.4). Its two qubits are *entangled*: the state cannot be written as a product $|a\rangle \otimes |b\rangle$. Measuring either qubit gives a random bit, but the two results always agree.

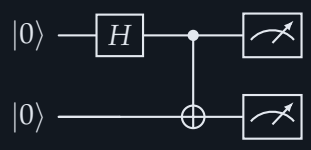


Figure 1.4: The Bell-state circuit: a Hadamard followed by a CNOT.

1.6.4 A first quantum advantage: Deutsch's algorithm

Suppose $f : \{0,1\} \to \{0,1\}$ is a black box and we want to know whether $f(0) = f(1)$. Classically we must evaluate f twice. A quantum computer can do it with one call to the oracle $U_f\,|x\rangle\,|y\rangle = |x\rangle\,|y \oplus f(x)\rangle$.

Phase kickback. Put the second qubit in $|-\rangle$. Then

$$U_f\,|x\rangle\,|-\rangle = |x\rangle\,\frac{|f(x)\rangle - |1 \oplus f(x)\rangle}{\sqrt{2}} = (-1)^{f(x)}\,|x\rangle\,|-\rangle\,, \tag{1.79}$$

because if $f(x) = 1$ the two terms swap and the sign flips. The function value has moved into a phase on the first qubit.

The algorithm. Start with $|0\rangle\,|1\rangle$ and apply $H \otimes H$, then U_f, then H on the first qubit:

$$|0\rangle\,|1\rangle \xrightarrow{H\otimes H} \frac{|0\rangle + |1\rangle}{\sqrt{2}}\,|-\rangle \xrightarrow{U_f} \frac{(-1)^{f(0)}\,|0\rangle + (-1)^{f(1)}\,|1\rangle}{\sqrt{2}}\,|-\rangle\,. \tag{1.80}$$

If $f(0) = f(1)$ the first qubit is $\pm|+\rangle$, and H turns it into $\pm|0\rangle$. If $f(0) \neq f(1)$ it is $\pm|-\rangle$, and H turns it into $\pm|1\rangle$. One measurement settles the question with certainty after a single query.

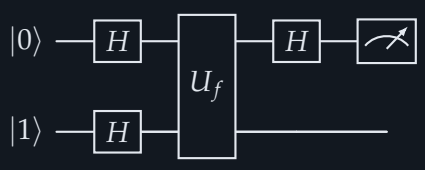


Figure 1.5: Deutsch's algorithm decides whether $f(0) = f(1)$ with one oracle query.

1.6.5 How big are quantum speedups for search and optimization?

Many guidance subproblems can be phrased as "search a huge set of options for a good one". For an unstructured search over N options, Grover's algorithm finds a marked item with about $\frac{\pi}{4}\sqrt{N}$ oracle queries, while a classical search needs about $N/2$ on average. For $N = 2^{40} \approx 1.1 \times 10^{12}$ that is roughly 8.2×10^5 quantum queries versus 5.5×10^{11} classical ones.

> **Reality check**: The speedup is quadratic, not exponential. Nielsen and Chuang prove that *no* quantum algorithm can do unstructured search with fewer than order $\sqrt{N}$ queries. So for generic hard optimization, quantum computers offer at most a quadratic speedup — and only once error-corrected hardware exists. Exponential speedups are known only for problems with special structure, such as factoring. Any claim of quantum advantage in this book must therefore be backed by a careful, problem-specific comparison with the best classical method, not by the general promise of 2^n amplitudes.

1.6.6 Variational algorithms: VQE and QAOA

Today's devices have tens to a few hundred noisy qubits. This is the *noisy intermediate-scale quantum* (NISQ) era. The algorithms designed for it are *variational*: a short parameterised circuit prepares a state $|\psi(\boldsymbol{\theta})\rangle$, the quantum device measures a cost, and a *classical* optimizer updates $\boldsymbol{\theta}$. The quantum computer is a co-processor inside a classical loop.

The variational principle. Let $\hat{H}$ be a Hermitian matrix (a "Hamiltonian") with eigenvalues $E_0 \le E_1 \le \cdots$ and orthonormal eigenvectors $|e_k\rangle$. Any normalized state can be written $|\psi\rangle = \sum_k c_k |e_k\rangle$ with $\sum_k |c_k|^2 = 1$. Then

$$\langle \hat{H} \rangle_\psi = \langle \psi | \hat{H} | \psi \rangle = \sum_k |c_k|^2 E_k \;\ge\; E_0 \sum_k |c_k|^2 = E_0. \tag{1.81}$$

So the expected value of $\hat{H}$ is always an upper bound on its smallest eigenvalue, with equality only for the ground state. The *variational quantum eigensolver* (VQE) minimizes $\langle \psi(\boldsymbol{\theta}) | \hat{H} | \psi(\boldsymbol{\theta}) \rangle$ over $\boldsymbol{\theta}$ to approach E_0.

From a decision problem to a Hamiltonian. Suppose a guidance or mission-planning choice is a vector of binary decisions $x_i \in \{0, 1\}$ with a quadratic cost $C(\boldsymbol{x}) = \boldsymbol{x}^{\mathsf{T}} \mathbf{Q} \boldsymbol{x} + \boldsymbol{c}^{\mathsf{T}} \boldsymbol{x}$ (a QUBO). Substitute $x_i = (1 - z_i)/2$ with $z_i \in \{+1, -1\}$. Since $Z|0\rangle = +|0\rangle$ and $Z|1\rangle = -|1\rangle$, replacing each z_i by the Pauli operator Z_i on qubit i gives a diagonal Hamiltonian

$$\hat{H}_C = \sum_{i<j} J_{ij} Z_i Z_j + \sum_i h_i Z_i + \text{const}, \tag{1.82}$$

whose ground state is the optimal decision vector. The *quantum approximate optimization algorithm* (QAOA) of Farhi, Goldstone and Gutmann (2014) prepares

$$|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle = \prod_{\ell=1}^{p} e^{-i\beta_\ell \hat{H}_B} e^{-i\gamma_\ell \hat{H}_C} \; |+\rangle^{\otimes n}, \qquad \hat{H}_B = \sum_i X_i, \tag{1.83}$$

and tunes the $2p$ angles to minimize $\langle \hat{H}_C \rangle$.

Worked example: Choosing one of two landing sites. Two landing sites have costs 3 and 5. Let $x_1, x_2 \in \{0, 1\}$ mean "choose site i", and add a penalty of weight 10 unless exactly one site is chosen:

$$C(x_1, x_2) = 3x_1 + 5x_2 + 10\,(x_1 + x_2 - 1)^2.$$

Checking all four options: $C(0,0) = 10$, $C(1,0) = 3$, $C(0,1) = 5$, $C(1,1) = 18$. The best choice is site 1.

Now convert. With $x_i = (1 - z_i)/2$ we get $x_1 + x_2 - 1 = -(z_1 + z_2)/2$, and because $z_i^2 = 1$,

$$(x_1 + x_2 - 1)^2 = \frac{(z_1 + z_2)^2}{4} = \frac{2 + 2z_1 z_2}{4} = \frac{1 + z_1 z_2}{2}.$$

So

$$C = \tfrac{3}{2}(1 - z_1) + \tfrac{5}{2}(1 - z_2) + 5(1 + z_1 z_2) = 9 - 1.5\,z_1 - 2.5\,z_2 + 5\,z_1 z_2,$$

and the Hamiltonian is $\hat{H}_C = 9I - 1.5Z_1 - 2.5Z_2 + 5Z_1 Z_2$. For site 1 ($x_1 = 1, x_2 = 0$, so $z_1 = -1$, $z_2 = +1$): $9 + 1.5 - 2.5 - 5 = 3$, matching $C(1,0)$. The ground state is $|10\rangle$. A two-variable problem is trivial, of course; the point is the recipe, which is identical for a thousand variables.

1.6.7 The NISQ reality and what it means for flight software

Every gate on today's hardware has a small error probability ε. If a circuit has G gates and errors are independent, the chance that no error occurs is

$$F \approx (1 - \varepsilon)^G \approx e^{-\varepsilon G}. \tag{1.84}$$

With $\varepsilon = 10^{-3}$: $G = 100$ gives $F \approx 0.90$, $G = 1000$ gives $F \approx 0.37$, and $G = 10\,000$ gives $F \approx 4.5 \times 10^{-5}$ (Fig. 1.6). Useful circuits must be short.

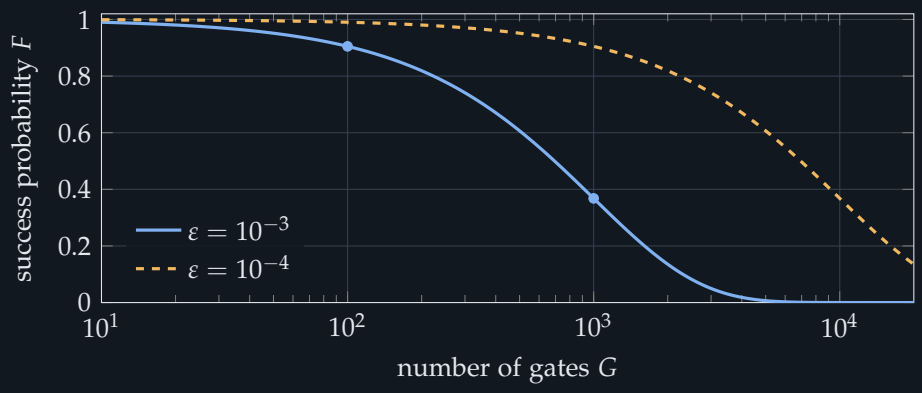


Figure 1.6: Approximate probability that a circuit runs without any gate error, $F \approx e^{-\varepsilon G}$. Even a tenfold improvement in gate error only moves the usable circuit depth by a factor of ten.

There is a second cost. A variational algorithm needs the *expected value* of the cost, and each run of the circuit returns only one random sample. To estimate an expectation with standard

deviation σ to precision ϵ we need about

$$N_{\text{shots}} \approx \frac{\sigma^2}{\epsilon^2} \tag{1.85}$$

repetitions; with $\sigma = 1$ and $\epsilon = 0.01$ that is 10^4 shots for *one* cost evaluation. The classical optimizer then needs many evaluations.

> **Reality check**: Timing budget. A launch-vehicle attitude loop typically runs at 50–100 Hz, giving 10–20 ms per cycle for navigation, guidance and control together. Even under optimistic assumptions about shot rate, a variational optimization with thousands of shots per evaluation and hundreds of evaluations takes seconds to minutes, before any network latency to a remote device. So in this book:
>
> - the **inner flight loop is always classical**;
> - quantum methods are used where time allows — pre-flight trajectory and mission planning, training of classifiers, offline dispersion analysis — and only kept if they beat a strong classical baseline;
> - every quantum component has a classical fallback that the vehicle can fly on alone.

1.6.8 What "demonstrable advantage" will mean

To keep ourselves honest we fix the test now. Let $\mathcal{Q}$ be a quantum or hybrid method and $\mathcal{B}$ the best classical method we can find for the same problem. Fix a solution quality target, for example an approximation ratio

$$r = \frac{C(\boldsymbol{x}_{\text{found}})}{C^\star} \leq 1.05, \tag{1.86}$$

where $C^\star$ is the optimum (or a duality-based lower bound, from the last section). We will say $\mathcal{Q}$ shows an advantage on a problem class only if, over a test set of instances, it reaches the target with higher probability or in less wall-clock time than $\mathcal{B}$, *including* all classical pre- and post-processing, queueing and communication time. If it does not, we use $\mathcal{B}$ and say so.

1.7 The four-phase architecture

The book builds the software in four phases. Each phase uses the previous one as its test bench, and each has a clear exit gate: we do not move on until the gate is passed.

Quantum methods appear in Phases 2 and 3 only, always alongside a classical method that does the same job.

1.8 Notation and conventions

We use the following conventions throughout the book.

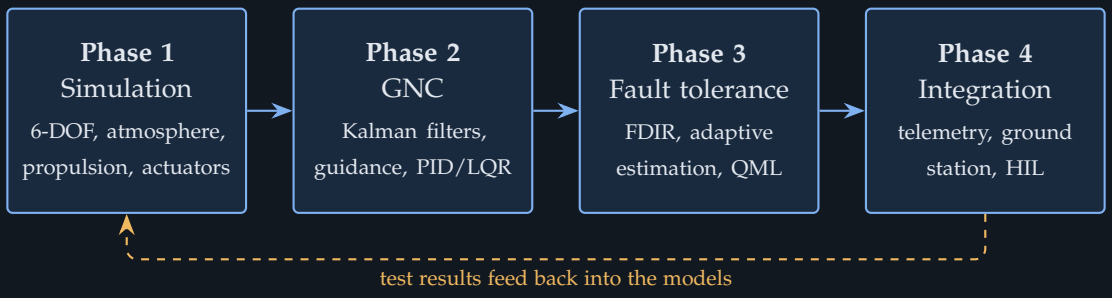


Figure 1.7: The four phases. Each phase is validated against the one before it.

Phase	Main mathematics	Example exit gate
1. Simulation	ODEs (1.2), rigid-body dynamics, numerical integration, Monte Carlo	Simulated ascent matches a reference case; energy and quaternion norm are conserved to tolerance when forces are switched off.
2. GNC	Kalman filtering, convex optimization, state feedback and frequency-domain margins	Closed-loop Monte Carlo meets orbit-insertion or landing accuracy with gain margin ≥ 6 dB and phase margin $\geq 30°$.
3. Fault tolerance	Hypothesis testing, multiple-model estimation, quantum and classical classifiers	Injected sensor and engine faults are detected within a set time with a bounded false-alarm rate.
4. Integration	Sampled-data control, real-time scheduling, communication protocols	Flight code on the target computer reproduces the Phase 2–3 results in hardware-in-the-loop runs, in real time.

Table 1.1: What each phase needs and how we know it is done. The numerical targets are typical starting values and will be tuned for each vehicle.

Symbol	Meaning
$a,\ \lambda$	scalars (italic)
$\boldsymbol{x},\ \boldsymbol{\omega}$	vectors (bold italic), column vectors by default
$\mathbf{A},\ \mathbf{J}$	matrices (bold upright)
$\mathbf{A}^{\mathsf{T}},\ \mathbf{A}^{-1}$	transpose, inverse
$\dot{x}$	time derivative $\mathrm{d}x/\mathrm{d}t$
$\hat{x}$	estimate of x; $\hat{x}^{-}$ estimate before the measurement update
x_k	value at time step k
$[\boldsymbol{v}]^B$	coordinates of vector $\boldsymbol{v}$ in frame B (Chapter 2)
$\mathbf{T}^{BI}$	transformation matrix from frame I to frame B
$\boldsymbol{q},\ \odot$	attitude quaternion and quaternion product
$\lVert\cdot\rVert_2$	Euclidean norm
$\mathcal{N}(\mu,\sigma^2)$	Gaussian distribution with mean μ and variance σ^2
$\lvert\psi\rangle,\ \langle\psi\rvert$	quantum state vector and its conjugate transpose
$\otimes$	tensor (Kronecker) product, used only for quantum systems
$U^{\dagger}$	conjugate transpose of a complex matrix
$X,\ Z,\ H$	Pauli-X, Pauli-Z and Hadamard gates

Units are SI throughout. Angles are in radians inside equations and in degrees in prose when that reads more naturally. Frames follow Zipfel: I is inertial, E Earth-fixed, B body, V velocity.

1.9 Road map of the book

Part I — Simulation foundation
Chapter 2 builds the mathematical toolkit: vectors, matrices, tensors, frames and coordinate systems. Chapter 3 derives the full equations of motion with variable mass and inertia. Chapter 4 and Chapter 5 model the atmosphere, aerodynamics, engines and gimbal actuators. Chapter 6 assembles the 6-DOF simulator and its Monte Carlo harness.

Part II — Guidance, navigation and control
Chapter 7 develops Kalman, extended and unscented filters for IMU, GPS and barometer fusion. Chapter 8 gives the quantum computing background. Chapter 9 formulates trajectory optimization, including lossless convexification. Chapter 10 tests QAOA and VQE against classical solvers. Chapter 11 designs PID, LQR and gain-scheduled controllers with robustness margins.

Part III — Fault tolerance and autonomy
Chapter 12 covers classical fault detection, isolation and recovery. Chapter 13 studies quantum machine learning for anomaly detection and dispersion prediction, with classical baselines.

Part IV — Ground and hardware integration
Chapter 14 builds telemetry and ground-station software. Chapter 15 moves the flight code onto real hardware in a hardware-in-the-loop rig. Chapter 16 brings everything together in verification, validation and flight qualification.

Summary

- A launch vehicle is a dynamical system $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u}, t)$ with a 14-element state. Its mass changes, which gives the rocket equation $\Delta v = v_e \ln(m_0/m_f)$ and the extra $\dot{\mathbf{J}}\boldsymbol{\omega}$ term in Euler's law.
- Simulation comes first. Step size is set by the fastest dynamics, and Monte Carlo error falls only as $1/\sqrt{N}$.
- Feedback multiplies model errors by the sensitivity $S = 1/(1 + PC)$ and can stabilise an aerodynamically unstable vehicle when $k_p > \mu_\alpha/\mu_c$.
- Estimation fuses prediction and measurement; precisions add, and the Kalman gain $K = \sigma_p^2/(\sigma_p^2 + \sigma_z^2)$ decides the balance.
- Guidance is optimization. Convex problems have no bad local minima; gimbal and glide-slope limits are second-order cones; the minimum-thrust limit is the main non-convexity.

- Quantum computers manipulate 2^n amplitudes but return only n bits. Speedups for generic search are at most quadratic, NISQ circuits must be short, and variational methods are slow. Quantum components therefore stay outside the real-time loop and must beat a strong classical baseline to earn their place.

Exercises

1.1 Rocket equation. A stage has $I_{sp} = 340\,\mathrm{s}$ and mass ratio 8. (a) Find Δv with no losses. (b) If it burns for 160 s straight up, find the gravity loss using $g = 9.81\,\mathrm{m/s^2}$ and the net Δv from (1.7).

1.2 Staging. Two identical stages each have mass ratio 4 and $v_e = 3000\,\mathrm{m/s}$. Compare their total Δv with a single stage of mass ratio $4 \times 4 = 16$. Explain why the answers are equal in this idealised case, and name one effect ignored here that makes real staging worthwhile.

1.3 Integrator stability. For $\dot{e} = -\lambda e$, show that the RK4 update (1.13) gives $e_{k+1} = R(h\lambda)\, e_k$ with $R(z) = 1 - z + \frac{z^2}{2} - \frac{z^3}{6} + \frac{z^4}{24}$. Find numerically the largest $h\lambda$ for which $|R| < 1$, and compare with forward Euler's limit of 2.

1.4 Quaternion drift. Write a short program that integrates (1.11) with forward Euler for a constant $\boldsymbol{\omega} = (0.1, 0.2, 0.3)\,\mathrm{rad/s}$ and $h = 0.01\,\mathrm{s}$ for 100 s, without re-normalising. Plot $\|\boldsymbol{q}\| - 1$. Repeat with RK4.

1.5 Pitch control. With $\mu_\alpha = 6/\mathrm{s^2}$ and $\mu_c = 10/\mathrm{s^2}$, find k_p and k_d giving $\omega_n = 4\,\mathrm{rad/s}$ and $\zeta = 0.8$. What is the largest initial pitch error the controller can correct without exceeding a $\pm 6^\circ$ gimbal?

1.6 Sensitivity. Show that $S + T = 1$. If $|PC| = 0.5$ at some frequency, what is $|S|$ at best and at worst? What does this say about disturbance rejection there?

1.7 Sensor fusion. Three independent altitude sources give 1500 m ($\sigma = 20\,\mathrm{m}$), 1520 m ($\sigma = 10\,\mathrm{m}$) and 1510 m ($\sigma = 5\,\mathrm{m}$). Use (1.57) twice to fuse them. Check that the order you fuse them in does not matter.

1.8 Steady-state Kalman gain. Derive (1.61). Then show that $K_\infty \to 1$ as $q/r \to \infty$ and $K_\infty \to 0$ as $q/r \to 0$, and explain each limit in words.

1.9 Convexity. Prove that the gimbal set (1.69) is convex directly from the definition, using the triangle inequality. Then show by a counterexample that $\{\boldsymbol{T} : \|\boldsymbol{T}\|_2 \geq T_{\min}\}$ is not convex.

1.10 Weak duality. For the problem "minimize x^2 subject to $1 - x \leq 0$", compute the dual function $g(\lambda)$, maximise it, and check that the best lower bound equals the optimal value 1.

1.11 Deutsch's algorithm. Work through Fig. 1.5 for all four functions $f : \{0,1\} \to \{0,1\}$ and confirm the measured bit is 0 exactly when $f(0) = f(1)$.

1.12 QUBO to Ising. Three candidate burn windows have costs 4, 2 and 7. Write a QUBO that chooses exactly one window, convert it to the form (1.82), and check the energies

of all eight bit strings.

1.13 Variational bound. For $\hat{H} = Z$ and $|\psi(\theta)\rangle = \cos\frac{\theta}{2}|0\rangle + \sin\frac{\theta}{2}|1\rangle$, show that $\langle\hat{H}\rangle = \cos\theta$, and check (1.81) for every θ.

1.14 Timing budget. Assume a device that runs one shot every 200 µs, an optimizer needing 300 cost evaluations, and 10^4 shots per evaluation. How long does one VQE run take? Compare it with a 10 ms control cycle and with a 10 min pre-launch planning window.

Reading for this chapter

- P. H. Zipfel, *Modeling and Simulation of Aerospace Vehicle Dynamics*, 2nd ed., AIAA, 2007: Chapter 1 and Appendix C.4.
- K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*, Princeton University Press, 2008: Chapter 1.
- R. R. Labbe, *Kalman and Bayesian Filters in Python*: Preface and Chapter 1.
- S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004: Chapter 1.
- M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th anniversary ed., Cambridge University Press, 2010: Chapter 1.
- B. Açıkmeşe and S. R. Ploen, "Convex programming approach to powered descent guidance for Mars landing," *Journal of Guidance, Control, and Dynamics*, 30(5), 2007.
- E. Farhi, J. Goldstone and S. Gutmann, "A quantum approximate optimization algorithm," arXiv:1411.4028, 2014.
- A. Peruzzo et al., "A variational eigenvalue solver on a photonic quantum processor," *Nature Communications* 5, 4213, 2014.
- J. Preskill, "Quantum computing in the NISQ era and beyond," *Quantum* 2, 79, 2018.

Chapter 2

Mathematical Foundations: Frames, Tensors and Coordinate Systems

> *A velocity of* $300\,\mathrm{m/s}$ *means nothing until you say: relative to what, and measured along which axes?*

Every number inside a flight simulator is one of three things: a plain number (mass, time), a vector (position, velocity, force), or a matrix-like object (an inertia tensor, a rotation). And every vector and matrix only has numbers once we have chosen *which frame* we are talking about and *which axes* we are measuring along. Most bugs in real flight software are not deep physics mistakes. They are a vector written in the wrong axes, a rotation applied in the wrong order, or a derivative taken in the wrong frame.

This chapter gives you the tools to never make those mistakes. We start from the very basics and build up:

1. vectors and matrices, and what they really mean (Section 2.1);
2. the linear algebra the whole book uses: eigenvalues, symmetric matrices, the matrix exponential, norms, and the complex version used in quantum computing (Section 2.2);
3. the difference between a *frame* and a *coordinate system*, and the frames a launch vehicle needs (Section 2.3);
4. how to convert a vector from one set of axes to another (Section 2.4);
5. Euler angles and why they fail on the launch pad, and quaternions, which do not (Sections 2.5 and 2.6);
6. how to take a time derivative when the axes themselves are rotating, which is where the Coriolis force comes from (Section 2.7);
7. tensors, with the inertia tensor as the main example (Section 2.8);
8. the shape of the Earth and how to turn latitude and longitude into a position vector (Section 2.9);
9. how numbers behave inside a computer (Section 2.10).

As in Chapter 1, every idea comes with a worked example that uses real numbers, so you can check each step with a calculator.

2.1 Vectors and matrices: what the numbers mean

2.1.1 A vector is an arrow, not a list of numbers

A *scalar* is a single number with a unit: a mass of 500 t, a time of 12 s. A *vector* has a size *and* a direction: the wind blowing east at 10 m/s, the thrust pushing the rocket upward, the position of the vehicle relative to the launch pad.

The important point is this. **The vector itself is a physical arrow. The list of numbers we write for it depends on the axes we choose.** The same wind can be written in many different ways, as Fig. 2.1 shows.

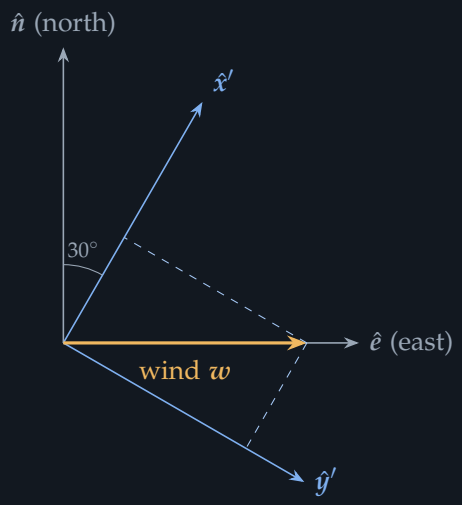


Figure 2.1: One wind vector $\boldsymbol{w}$, two sets of axes. Along east and north it is $(10,\ 0)$ m/s (east, north). Along the rotated axes $\hat{x}'$ (pointing 30° east of north) and $\hat{y}'$ it is $(5,\ 8.66)$ m/s. The arrow has not changed; only its description has.

Worked example: One wind, two descriptions. The wind blows toward the east at 10 m/s. A rocket on the pad is pointed with its x'-axis at a heading of 30° (that is, 30° east of north), and its y'-axis 90° to the right of that. The component of the wind along an axis is the wind speed times the cosine of the angle between them:

$$w_{x'} = 10\cos 60^\circ = 5.00\,\text{m/s}, \qquad w_{y'} = 10\cos 30^\circ = 8.66\,\text{m/s}.$$

Check: $\sqrt{5^2 + 8.66^2} = \sqrt{25 + 75} = 10$. The length is the same in both descriptions, as it must be.

We write the physical vector in bold, $\boldsymbol{v}$, and its list of components in a particular coordinate

system A as $[\boldsymbol{v}]^A$, a column of three numbers:

$$[\boldsymbol{v}]^A = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix}, \qquad \boldsymbol{v} = v_1\,\hat{\boldsymbol{a}}_1 + v_2\,\hat{\boldsymbol{a}}_2 + v_3\,\hat{\boldsymbol{a}}_3, \tag{2.1}$$

where $\hat{\boldsymbol{a}}_1, \hat{\boldsymbol{a}}_2, \hat{\boldsymbol{a}}_3$ are the unit vectors along the axes of A. The hat means "length one".

2.1.2 The operations you need

Adding and scaling. Vectors add tip-to-tail, and in components you simply add matching entries. Multiplying by a number stretches the arrow. These are what let us write, for example, total force = thrust + drag + weight.

The dot product. For two vectors in the same coordinate system,

$$\boldsymbol{a} \cdot \boldsymbol{b} = a_1 b_1 + a_2 b_2 + a_3 b_3 = \|\boldsymbol{a}\|\ \|\boldsymbol{b}\| \cos\gamma, \tag{2.2}$$

where γ is the angle between them and $\|\boldsymbol{a}\| = \sqrt{\boldsymbol{a} \cdot \boldsymbol{a}}$ is the length. The dot product answers two everyday questions:

- *What is the angle between two directions?* $\cos\gamma = \dfrac{\boldsymbol{a} \cdot \boldsymbol{b}}{\|\boldsymbol{a}\|\,\|\boldsymbol{b}\|}$. If $\boldsymbol{a} \cdot \boldsymbol{b} = 0$ the vectors are perpendicular.
- *How much of $\boldsymbol{a}$ points along a unit direction $\hat{\boldsymbol{u}}$?* The answer is $\boldsymbol{a} \cdot \hat{\boldsymbol{u}}$. This is exactly what we did with the wind: each component is a dot product with a unit axis vector.

In matrix notation, with column vectors, $\boldsymbol{a} \cdot \boldsymbol{b} = \boldsymbol{a}^{\mathsf{T}} \boldsymbol{b}$.

The cross product. This one only exists in three dimensions:

$$\boldsymbol{a} \times \boldsymbol{b} = \begin{bmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{bmatrix}, \qquad \|\boldsymbol{a} \times \boldsymbol{b}\| = \|\boldsymbol{a}\|\ \|\boldsymbol{b}\| \sin\gamma. \tag{2.3}$$

The result is perpendicular to both $\boldsymbol{a}$ and $\boldsymbol{b}$, with direction given by the *right-hand rule*: point the fingers of your right hand along $\boldsymbol{a}$, curl them toward $\boldsymbol{b}$, and your thumb points along $\boldsymbol{a} \times \boldsymbol{b}$. Note that the order matters: $\boldsymbol{b} \times \boldsymbol{a} = -\boldsymbol{a} \times \boldsymbol{b}$.

The cross product is how we compute a *moment* (a twisting effect): a force $\boldsymbol{F}$ applied at a point $\boldsymbol{r}$ from the centre of mass produces the moment $\boldsymbol{M} = \boldsymbol{r} \times \boldsymbol{F}$.

> **Worked example**: The moment from a gimballed engine. Use the body axes of Fig. 2.2. The engine is $\ell = 15\,\text{m}$ behind the centre of gravity, so $[\boldsymbol{r}]^B = (-15,\ 0,\ 0)\,\text{m}$. The thrust is $T = 900\,\text{kN}$, tilted by $\delta = 2^\circ$ so that part of it pushes the tail toward $+z_B$:

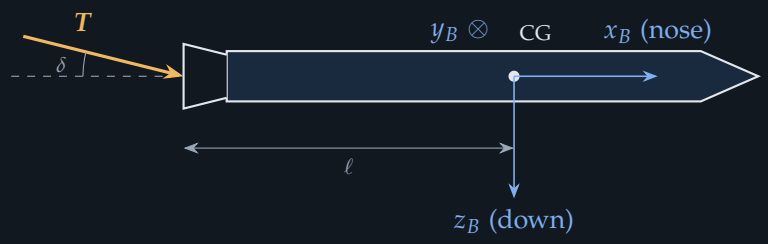


Figure 2.2: Body axes: x_B out of the nose, y_B out of the right side (into the page, shown ⊗), z_B completing a right-handed set, pointing "down" in normal flight. The engine sits a distance ℓ behind the centre of gravity, and the gimbal tilts the thrust by δ. (The angle is exaggerated.)

$[\boldsymbol{T}]^B = T\,(\cos\delta,\ 0,\ \sin\delta)$. Using (2.3) with $\boldsymbol{a} = \boldsymbol{r}$, $\boldsymbol{b} = \boldsymbol{T}$:

$$\boldsymbol{M} = \boldsymbol{r} \times \boldsymbol{T} = \begin{bmatrix} 0 \cdot T\sin\delta - 0 \cdot 0 \\ 0 \cdot T\cos\delta - (-\ell)\,T\sin\delta \\ (-\ell)\cdot 0 - 0 \cdot T\cos\delta \end{bmatrix} = \begin{bmatrix} 0 \\ \ell\,T\sin\delta \\ 0 \end{bmatrix}.$$

Numerically, $M_y = 15 \times 900\,000 \times \sin 2^\circ = 15 \times 900\,000 \times 0.0349 \approx 471\,\text{kN m}$. The moment is about the y_B-axis (pitch), and positive, which by the right-hand rule about y_B means *nose up*: pushing the tail down tips the nose up. This is the same relation we wrote in Chapter 1 as $M \approx T\ell\delta$, now with the direction worked out properly.

The cross product as a matrix. It will be very useful to write "$\boldsymbol{a}\times$" as a matrix that multiplies $\boldsymbol{b}$. Reading off (2.3):

$$\boldsymbol{a} \times \boldsymbol{b} = [\boldsymbol{a}\times]\,\boldsymbol{b}, \qquad [\boldsymbol{a}\times] = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix}. \tag{2.4}$$

This matrix is *skew-symmetric*: $[\boldsymbol{a}\times]^{\mathsf{T}} = -[\boldsymbol{a}\times]$. We used exactly this property in Chapter 1 to show that quaternions keep their length.

2.1.3 Matrices as machines that transform vectors

A 3×3 matrix $\mathbf{A}$ takes a vector $\boldsymbol{x}$ and produces a new vector $\boldsymbol{y} = \mathbf{A}\boldsymbol{x}$, with $y_i = \sum_j A_{ij}x_j$. In this book a matrix almost always means one of three things:

- a **change of axes**: the same vector, rewritten in different coordinates (Section 2.4);
- a **physical relation** between two vectors: angular momentum = inertia tensor × angular velocity (Section 2.8);
- a **system matrix**: $\dot{\boldsymbol{x}} = \mathbf{A}\boldsymbol{x}$ describing how a state evolves (Section 2.2).

The rules you need: matrix products are done row-times-column; $(\mathbf{AB})^{\mathsf{T}} = \mathbf{B}^{\mathsf{T}}\mathbf{A}^{\mathsf{T}}$; the identity $\mathbf{I}$ has ones on the diagonal and leaves every vector unchanged; and the inverse $\mathbf{A}^{-1}$, if it exists,

undoes $\mathbf{A}$: $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$. The inverse exists exactly when the *determinant* is not zero. For a 2×2 matrix,

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc, \qquad \begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}. \tag{2.5}$$

Geometrically, $|\det \mathbf{A}|$ is how much $\mathbf{A}$ stretches volumes; a determinant of zero means $\mathbf{A}$ squashes space flat and cannot be undone.

> **Key idea.** Matrix multiplication is *not* commutative: in general $\mathbf{AB} \neq \mathbf{BA}$. For rotations this has a physical meaning. Turning a rocket 90° in pitch and then 90° in roll leaves it pointing somewhere different from roll-then-pitch. We will see this with numbers in Section 2.4.

2.2 The linear algebra the whole book uses

2.2.1 Eigenvalues and eigenvectors

Most vectors change direction when a matrix acts on them. A few special ones do not; they only get stretched.

Definition 2.1. A non-zero vector $\boldsymbol{v}$ is an *eigenvector* of a square matrix $\mathbf{A}$, with *eigenvalue* λ, if

$$\mathbf{A}\boldsymbol{v} = \lambda \boldsymbol{v}. \tag{2.6}$$

How to find them. Rewrite the definition as $(\mathbf{A} - \lambda \mathbf{I})\boldsymbol{v} = \mathbf{0}$. A non-zero $\boldsymbol{v}$ can solve this only if $\mathbf{A} - \lambda \mathbf{I}$ squashes space flat, that is,

$$\det(\mathbf{A} - \lambda \mathbf{I}) = 0. \tag{2.7}$$

This is a polynomial in λ, the *characteristic polynomial*. Its roots are the eigenvalues.

Why we care: eigenvalues are the poles. Write the uncontrolled pitch dynamics from Chapter 1, $\ddot{\theta} = \mu_\alpha \theta$, as two first-order equations using the state $\boldsymbol{x} = (\theta,\ \dot{\theta})$:

$$\dot{\boldsymbol{x}} = \mathbf{A}\boldsymbol{x}, \qquad \mathbf{A} = \begin{bmatrix} 0 & 1 \\ \mu_\alpha & 0 \end{bmatrix}. \tag{2.8}$$

Then

$$\det(\mathbf{A} - \lambda \mathbf{I}) = \det \begin{bmatrix} -\lambda & 1 \\ \mu_\alpha & -\lambda \end{bmatrix} = \lambda^2 - \mu_\alpha = 0 \quad \Longrightarrow \quad \lambda = \pm\sqrt{\mu_\alpha}.$$

With $\mu_\alpha = 4$ these are ± 2: exactly the poles we found with the transfer function in Chapter 1. With the PD controller of that chapter, the closed-loop matrix becomes

$$\mathbf{A}_{\text{cl}} = \begin{bmatrix} 0 & 1 \\ -9 & -4.2 \end{bmatrix}, \qquad \lambda^2 + 4.2\lambda + 9 = 0 \quad \Longrightarrow \quad \lambda = -2.1 \pm 2.14i.$$

> **Key idea.** For a system $\dot{\boldsymbol{x}} = \mathbf{A}\boldsymbol{x}$, the eigenvalues of $\mathbf{A}$ are the poles. The system is stable when every eigenvalue has a negative real part. Transfer functions and state-space matrices are two views of the same thing.

2.2.2 Symmetric matrices

A matrix is *symmetric* if $\mathbf{A}^\mathsf{T} = \mathbf{A}$. Two of the most important matrices in this book are symmetric: the inertia tensor (Section 2.8) and the covariance matrix of a Kalman filter (Chapter 7). Symmetric matrices are the best-behaved matrices there are.

Theorem 2.2 (Spectral theorem, real symmetric case)**.** *If $\mathbf{A}$ is real and symmetric, then all its eigenvalues are real, and eigenvectors belonging to different eigenvalues are perpendicular. Moreover $\mathbf{A}$ can be written as*

$$\mathbf{A} = \mathbf{Q}\blacksquare\mathbf{Q}^\mathsf{T}, \tag{2.9}$$

where the columns of $\mathbf{Q}$ are orthonormal eigenvectors and $\blacksquare$ is diagonal with the eigenvalues.

Proof of the perpendicular part. Let $\mathbf{A}\boldsymbol{u} = \lambda\boldsymbol{u}$ and $\mathbf{A}\boldsymbol{v} = \mu\boldsymbol{v}$ with $\lambda \neq \mu$. Then

$$\lambda\, \boldsymbol{v}^\mathsf{T}\boldsymbol{u} = \boldsymbol{v}^\mathsf{T}(\mathbf{A}\boldsymbol{u}) = (\mathbf{A}^\mathsf{T}\boldsymbol{v})^\mathsf{T}\boldsymbol{u} = (\mathbf{A}\boldsymbol{v})^\mathsf{T}\boldsymbol{u} = \mu\, \boldsymbol{v}^\mathsf{T}\boldsymbol{u}.$$

So $(\lambda - \mu)\, \boldsymbol{v}^\mathsf{T}\boldsymbol{u} = 0$, and since $\lambda \neq \mu$, $\boldsymbol{v}^\mathsf{T}\boldsymbol{u} = 0$. □

In words: *every symmetric matrix is just a stretch along some set of perpendicular axes*. Find those axes and the matrix becomes diagonal. For the inertia tensor, those axes are the *principal axes* of the body.

Definition 2.3. A symmetric matrix $\mathbf{A}$ is *positive definite* if $\boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x} > 0$ for every non-zero $\boldsymbol{x}$. Equivalently, all its eigenvalues are positive.

Inertia tensors and covariance matrices are always positive definite (or at least semi-definite): a body cannot have negative rotational inertia, and an uncertainty cannot have negative variance. If your Kalman filter ever produces a covariance matrix with a negative eigenvalue, something has gone numerically wrong.

2.2.3 The matrix exponential

For one number, $\dot{x} = ax$ has the solution $x(t) = e^{at}x(0)$. For a system $\dot{\boldsymbol{x}} = \mathbf{A}\boldsymbol{x}$ the solution looks the same:

$$\boldsymbol{x}(t) = e^{\mathbf{A}t}\,\boldsymbol{x}(0), \qquad e^{\mathbf{A}t} = \mathbf{I} + \mathbf{A}t + \frac{(\mathbf{A}t)^2}{2!} + \frac{(\mathbf{A}t)^3}{3!} + \cdots \tag{2.10}$$

The series is the same as for the ordinary exponential, with $\mathbf{A}$ in place of a. Differentiating term by term gives $\frac{\mathrm{d}}{\mathrm{d}t}e^{\mathbf{A}t} = \mathbf{A}e^{\mathbf{A}t}$, so (2.10) really does solve the equation.

Worked example: Where the Kalman filter's prediction comes from. A vehicle moving in a straight line at constant speed has state $\boldsymbol{x} = (\text{position},\ \text{velocity})$ and

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \qquad \mathbf{A}^2 = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}.$$

Because $\mathbf{A}^2 = \mathbf{0}$, every term after the second in (2.10) vanishes, and over a time step Δt

$$e^{\mathbf{A}\Delta t} = \mathbf{I} + \mathbf{A}\Delta t = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}.$$

So the new position is old position $+\ \Delta t \times$ velocity, and the velocity stays the same. This is exactly the "predict" step of the g–h filter in Chapter 1, and it is the matrix $\mathbf{F}$ of the Kalman filter for this model.

2.2.4 Norms and conditioning

A *norm* measures the size of a vector. Three are common:

$$\|\boldsymbol{x}\|_2 = \sqrt{\textstyle\sum_i x_i^2}, \qquad \|\boldsymbol{x}\|_1 = \textstyle\sum_i |x_i|, \qquad \|\boldsymbol{x}\|_\infty = \max_i |x_i|. \tag{2.11}$$

The Euclidean norm $\|\cdot\|_2$ is ordinary length. The other two appear in optimization: $\|\cdot\|_1$ for fuel use (the sum of thrust magnitudes), $\|\cdot\|_\infty$ for "the largest error anywhere". One inequality is used constantly, the *Cauchy–Schwarz inequality*:

$$\left|\boldsymbol{a}^{\mathsf{T}}\boldsymbol{b}\right| \le \|\boldsymbol{a}\|_2\ \|\boldsymbol{b}\|_2, \tag{2.12}$$

which is just $|\cos\gamma| \le 1$ from (2.2).

Every matrix can be written as a rotation, a stretch along perpendicular axes, and another rotation. This is the *singular value decomposition* $\mathbf{A} = \mathbf{U}\blacksquare\mathbf{V}^{\mathsf{T}}$, with the stretches $\sigma_1 \ge \sigma_2 \ge \cdots \ge 0$ on the diagonal of $\blacksquare$. The ratio

$$\kappa(\mathbf{A}) = \frac{\sigma_{\max}}{\sigma_{\min}} \tag{2.13}$$

is the *condition number*. When we solve $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ on a computer, a small relative error in $\boldsymbol{b}$ can grow by up to a factor of κ in $\boldsymbol{x}$.

Worked example: An almost-singular system.

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 1 & 1.0001 \end{bmatrix}.$$

This matrix is symmetric and positive definite, so its singular values equal its eigenvalues, about 2.00005 and 0.00005. The condition number is about 40 000. An error in the fifth significant figure of $\boldsymbol{b}$ can wipe out the first significant figure of $\boldsymbol{x}$. Matrices like this appear when two sensors measure almost the same thing, which is why navigation filters need care (Chapter 7).

2.2.5 Complex vectors: the same ideas in quantum notation

Quantum computing (Chapter 8) uses vectors with *complex* entries. Almost everything above carries over if you replace "transpose" by "conjugate transpose", written with a dagger: $\mathbf{A}^\dagger = \overline{\mathbf{A}}^\mathsf{T}$. The table is worth memorising, because it lets you reuse everything you know.

Idea	Real (flight dynamics)	Complex (quantum)
inner product	$\boldsymbol{a}^\mathsf{T}\boldsymbol{b}$	$\langle a \vert b \rangle = \boldsymbol{a}^\dagger \boldsymbol{b}$
length-preserving matrix	orthogonal: $\mathbf{R}^\mathsf{T}\mathbf{R} = \mathbf{I}$	unitary: $U^\dagger U = I$
"nice" matrix with real eigenvalues	symmetric: $\mathbf{A}^\mathsf{T} = \mathbf{A}$	Hermitian: $H^\dagger = H$
example	rotation matrix, inertia tensor	quantum gate, Hamiltonian

A rotation matrix for a rocket and a quantum gate for a qubit are the same kind of object: a matrix that changes the description without changing the length.

The tensor (Kronecker) product. To describe two qubits together we combine their vectors with the Kronecker product. For a 2×2 matrix $\mathbf{A}$ and any matrix $\mathbf{B}$,

$$\mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} a_{11}\mathbf{B} & a_{12}\mathbf{B} \\ a_{21}\mathbf{B} & a_{22}\mathbf{B} \end{bmatrix}, \qquad \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} \otimes \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} = \begin{bmatrix} a_1 b_1 \\ a_1 b_2 \\ a_2 b_1 \\ a_2 b_2 \end{bmatrix}. \tag{2.14}$$

Sizes multiply: two 2-vectors give a 4-vector, and n qubits give a 2^n-vector. A useful rule is $(\mathbf{A} \otimes \mathbf{B})(\mathbf{C} \otimes \mathbf{D}) = (\mathbf{AC}) \otimes (\mathbf{BD})$: applying a gate to each qubit separately is the same as applying their Kronecker product to the pair.

2.3 Frames and coordinate systems

2.3.1 Two words that are not the same

Everyday language uses "frame" and "coordinate system" as if they meant the same thing. In flight dynamics, following Zipfel, we keep them apart, because mixing them up causes real errors.

A frame is a rigid body of reference: a set of points that do not move relative to each other. "The Earth", "the rocket" and "the distant stars" are frames. A frame tells you *what counts as standing still*. Velocity, acceleration and time derivatives only make sense *relative to a frame*.

A coordinate system is a choice of origin and three perpendicular unit axes, fixed in some frame, used to write vectors as lists of numbers. A coordinate system tells you *which numbers to write down*.

Worked example: A point on the launch pad. Take a point on the launch pad at Cape Canaveral, latitude 28.6°. Relative to the *Earth frame* it is not moving: its velocity is zero. Relative to an *inertial frame* (one that does not rotate with the Earth) it is carried around by the Earth's rotation at

$$v = \omega_E R \cos\varphi \approx 7.292 \times 10^{-5} \times 6\,383\,000 \times \cos 28.6^\circ \approx 408\,\mathrm{m/s}$$

toward the east. Same point, two frames, two completely different velocities. Neither is wrong. This is free speed for a rocket launched eastward, and one reason launch sites are placed close to the equator, where the figure is 465 m/s.

The general rule: **the frame decides the physics (what the velocity is); the coordinate system decides the bookkeeping (which numbers describe it).** We can write the inertial velocity of the rocket in Earth-fixed axes, or its Earth-relative velocity in body axes. Both are legitimate, and a simulator does both constantly.

2.3.2 The frames of a launch vehicle

Frame (symbol)	Origin and axes	What we use it for
Inertial, ECI (I)	Earth's centre; z along the spin axis, x toward a fixed direction in the sky; does not rotate	Newton's law holds here; orbital mechanics; navigation integrates here
Earth-fixed, ECEF (E)	Earth's centre; z along the spin axis, x through latitude 0°, longitude 0°; rotates with the Earth	Launch site and landing site positions; GPS outputs
Local level, NED (G)	A point on the Earth's surface; x north, y east, z down	Altitude, heading, flight-path angle; what a human understands
Body (B)	Vehicle centre of mass; x out of the nose, y right, z down (Fig. 2.2)	Engine and fin forces; IMU measurements; inertia tensor
Velocity, or wind (V)	Vehicle centre of mass; x along the velocity relative to the air	Lift and drag; angle of attack

Table 2.1: The main frames and coordinate systems in this book. In practice each frame comes with its own natural coordinate system, so we usually use the same letter for both.

Reality check: Is the ECI frame really inertial?. Not exactly. The Earth's centre accelerates toward the Sun at about $(29.8 \times 10^3)^2 / 1.496 \times 10^{11} \approx 5.9 \times 10^{-3}\,\mathrm{m/s^2}$. But the Sun pulls on the rocket almost exactly as hard, so the rocket and the Earth fall toward the Sun together and the difference is tiny. For the few minutes of an ascent or a landing, treating ECI as inertial is an excellent approximation.

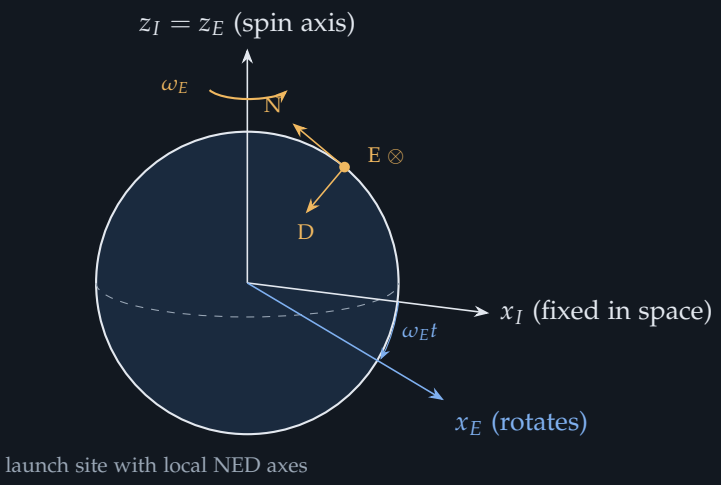


Figure 2.3: Inertial axes stay fixed in space; Earth-fixed axes turn with the Earth at $\omega_E = 7.292\,115\,9 \times 10^{-5}\,\mathrm{rad/s}$ (one turn per sidereal day); local north–east–down axes sit on the surface at the launch site.

2.4 Changing coordinates: the direction cosine matrix

2.4.1 Deriving the transformation

Suppose we know the components of a vector in coordinate system A and want them in coordinate system B. Start from (2.1): $\boldsymbol{v} = v_1^A \hat{\boldsymbol{a}}_1 + v_2^A \hat{\boldsymbol{a}}_2 + v_3^A \hat{\boldsymbol{a}}_3$. The i-th component in B is the dot product with the i-th axis of B:

$$v_i^B = \hat{\boldsymbol{b}}_i \cdot \boldsymbol{v} = \sum_{j=1}^{3} (\hat{\boldsymbol{b}}_i \cdot \hat{\boldsymbol{a}}_j)\, v_j^A. \tag{2.15}$$

Collect the dot products into a matrix and this becomes one matrix multiplication:

$$\boxed{[\boldsymbol{v}]^B = \mathbf{T}^{BA}\,[\boldsymbol{v}]^A, \qquad T_{ij}^{BA} = \hat{\boldsymbol{b}}_i \cdot \hat{\boldsymbol{a}}_j = \cos(\text{angle between } \hat{\boldsymbol{b}}_i \text{ and } \hat{\boldsymbol{a}}_j).} \tag{2.16}$$

Each entry is a cosine of an angle between two axes, which is why $\mathbf{T}^{BA}$ is called the *direction cosine matrix* (DCM). Read the superscript from right to left: $\mathbf{T}^{BA}$ takes A coordinates and produces B coordinates.

2.4.2 Properties that make life easy

Proposition 2.4. *Every direction cosine matrix is* orthogonal, $\mathbf{T}^{\mathsf{T}}\mathbf{T} = \mathbf{I}$, *and has* $\det \mathbf{T} = +1$. *Hence its inverse is just its transpose:*

$$\mathbf{T}^{AB} = \left(\mathbf{T}^{BA}\right)^{-1} = \left(\mathbf{T}^{BA}\right)^{\mathsf{T}}. \tag{2.17}$$

Proof. The i-th row of $\mathbf{T}^{BA}$ is $\hat{\boldsymbol{b}}_i$ written in A coordinates. The rows are therefore three perpendicular unit vectors, so the dot product of row i with row k is 1 if $i = k$ and 0 otherwise. That is exactly $\mathbf{T}\mathbf{T}^{\mathsf{T}} = \mathbf{I}$, and for square matrices this implies $\mathbf{T}^{\mathsf{T}}\mathbf{T} = \mathbf{I}$ too. The determinant is $+1$ rather than -1 because both coordinate systems are right-handed. □

This is a gift: inverting a general 3×3 matrix takes work, but inverting a rotation is just a transpose. Orthogonal matrices also preserve lengths and angles: $\|\mathbf{T}v\|^2 = v^\mathsf{T}\mathbf{T}^\mathsf{T}\mathbf{T}v = v^\mathsf{T}v$.

Chaining. To go from A to C through B, apply one after the other:

$$\mathbf{T}^{CA} = \mathbf{T}^{CB}\,\mathbf{T}^{BA}. \tag{2.18}$$

The inner letters match and "cancel", which makes it easy to check you have the order right.

2.4.3 The three elementary rotations

Any change of axes can be built from rotations about one axis at a time. If the new axes are the old ones turned by an angle α about the old x-, y- or z-axis, the coordinate transformations are

$$\mathbf{R}_1(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\alpha & \sin\alpha \\ 0 & -\sin\alpha & \cos\alpha \end{bmatrix},\ \mathbf{R}_2(\alpha) = \begin{bmatrix} \cos\alpha & 0 & -\sin\alpha \\ 0 & 1 & 0 \\ \sin\alpha & 0 & \cos\alpha \end{bmatrix},\ \mathbf{R}_3(\alpha) = \begin{bmatrix} \cos\alpha & \sin\alpha & 0 \\ -\sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{2.19}$$

The axis you rotate about keeps its component (the row and column with the 1). A check on the signs: the wind example of Fig. 2.1 is a rotation by $\psi = 30^\circ$ about the vertical. With (north, east) components $(0, 10)$, the top-left 2×2 block of $\mathbf{R}_3(30^\circ)$ gives $(10\sin 30^\circ,\ 10\cos 30^\circ) = (5,\ 8.66)$, as before.

> **Worked example**: Order matters. Take 90° about x followed by 90° about the new y, and the other way round. Rounding $\cos 90^\circ = 0$ and $\sin 90^\circ = 1$:
>
> $$\mathbf{R}_2(90^\circ)\mathbf{R}_1(90^\circ) = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}, \qquad \mathbf{R}_1(90^\circ)\mathbf{R}_2(90^\circ) = \begin{bmatrix} 0 & 0 & -1 \\ 1 & 0 & 0 \\ 0 & -1 & 0 \end{bmatrix}.$$
>
> Different matrices, so different final orientations. Large rotations do not add up like vectors. (Small rotations almost do, which is why linearised models work.)

2.5 Euler angles, and why they fail on the launch pad

2.5.1 Yaw, pitch and roll

The most intuitive way to describe the attitude of a vehicle is with three angles, applied in a fixed order to go from the local-level axes G (north, east, down) to the body axes B. The aerospace standard is the "3-2-1" sequence:

1. turn by the **yaw** (heading) angle ψ about the down axis (z);
2. then by the **pitch** angle θ about the new right axis (y);
3. then by the **roll** angle ϕ about the new nose axis (x).

Chaining the elementary rotations, with the first rotation on the right:

$$\mathbf{T}^{BG} = \mathbf{R}_1(\phi)\,\mathbf{R}_2(\theta)\,\mathbf{R}_3(\psi). \tag{2.20}$$

Multiplying out, and writing c for cosine and s for sine,

$$\mathbf{T}^{BG} = \begin{bmatrix} c\theta\,c\psi & c\theta\,s\psi & -s\theta \\ s\phi\,s\theta\,c\psi - c\phi\,s\psi & s\phi\,s\theta\,s\psi + c\phi\,c\psi & s\phi\,c\theta \\ c\phi\,s\theta\,c\psi + s\phi\,s\psi & c\phi\,s\theta\,s\psi - s\phi\,c\psi & c\phi\,c\theta \end{bmatrix}. \tag{2.21}$$

Going back from the matrix to the angles uses three of its entries:

$$\theta = -\arcsin T_{13}, \qquad \psi = \operatorname{atan2}(T_{12},\, T_{11}), \qquad \phi = \operatorname{atan2}(T_{23},\, T_{33}). \tag{2.22}$$

(The two-argument arctangent $\operatorname{atan2}(y, x)$ returns the angle in the correct quadrant; always use it instead of $\arctan(y/x)$.)

Worked example: A rocket shortly after lift-off. Heading $\psi = 30°$, pitch $\theta = 80°$ (nearly vertical), roll $\phi = 5°$. Evaluating (2.21) to four decimals:

$$\mathbf{T}^{BG} \approx \begin{bmatrix} 0.1504 & 0.0868 & -0.9848 \\ -0.4238 & 0.9056 & 0.0151 \\ 0.8932 & 0.4151 & 0.1730 \end{bmatrix}.$$

The first row is the nose direction in (north, east, down) coordinates: mostly "up" (-0.98 along down), tilted a little north and east. Recovering the angles with (2.22): $\theta = -\arcsin(-0.9848) = 80°$, $\psi = \operatorname{atan2}(0.0868, 0.1504) = 30°$, $\phi = \operatorname{atan2}(0.0151, 0.1730) = 5°$. Everything is recovered.

2.5.2 Gimbal lock

Now stand the rocket exactly upright: $\theta = 90°$, so $c\theta = 0$ and $s\theta = 1$. Look at rows 2 and 3 of (2.21) and use the angle-difference formulas $\sin(\phi - \psi) = s\phi\,c\psi - c\phi\,s\psi$ and $\cos(\phi - \psi) = c\phi\,c\psi + s\phi\,s\psi$:

$$\mathbf{T}^{BG}\big|_{\theta=90°} = \begin{bmatrix} 0 & 0 & -1 \\ \sin(\phi-\psi) & \cos(\phi-\psi) & 0 \\ \cos(\phi-\psi) & -\sin(\phi-\psi) & 0 \end{bmatrix}. \tag{2.23}$$

The matrix only depends on the *difference* $\phi - \psi$. Yaw and roll have become the same motion (both spin the rocket about its vertical axis), and no amount of looking at the matrix can separate them. In (2.22), ψ and ϕ become $\operatorname{atan2}(0, 0)$, which is undefined. This is *gimbal lock*.

The problem also appears in the equations that turn body angular rates (p, q, r) (what the gyroscopes measure) into Euler angle rates:

$$\begin{aligned} \dot\phi &= p + (q\sin\phi + r\cos\phi)\tan\theta, \\ \dot\theta &= q\cos\phi - r\sin\phi, \\ \dot\psi &= \frac{q\sin\phi + r\cos\phi}{\cos\theta}. \end{aligned} \tag{2.24}$$

At $\theta = 90°$, $\tan\theta$ and $1/\cos\theta$ are infinite. A simulator integrating these equations will blow up as the rocket passes through vertical — which every rocket does at lift-off.

> **Reality check.** Gimbal lock is not a physical problem; the rocket is perfectly happy pointing straight up. It is a problem with the *description*. Any description of attitude using only three numbers has a singularity somewhere; this is a mathematical fact about rotations. The cure is to use four numbers with one constraint: the quaternion.

2.6 Quaternions

In Section 2.5 we saw the problem with Euler angles: when the rocket points straight up, two of the three angles become the same motion and the equations divide by zero. Every rocket starts pointing straight up, so we need something better.

The fix is the *quaternion*. The name sounds exotic, but the idea is simple:

> **Key idea.** A quaternion is four numbers that store *one axis* and *one angle*. That is all. Everything else in this section is about how to store them cleverly, how to combine two of them, and how to use them to turn vectors.

We take it one step at a time.

2.6.1 Step 1: any turn is one turn about one axis

Pick up a book from a table. Turn it over, spin it, tilt it — do anything you like. However complicated your movement was, there is always a single straight line (an *axis*) such that one turn about that line, by one angle, would take the book from where it started to where it ended. This is *Euler's rotation theorem*.

So every orientation change can be described by just two things:

- an axis, written as a unit vector $\hat{\boldsymbol{n}}$ (an arrow of length 1), and
- an angle α: how far to turn about that axis, using the right-hand rule (thumb along $\hat{\boldsymbol{n}}$, fingers curl in the direction of turning).

2.6.2 Step 2: storing the axis and angle as four numbers

The obvious idea would be to store the three numbers of the axis and the one number of the angle. That almost works, but it has two annoying problems: when the angle is zero, the axis can be anything (so the description is not unique), and an angle of $0°$ and an angle of $360°$ look different even though they are the same orientation.

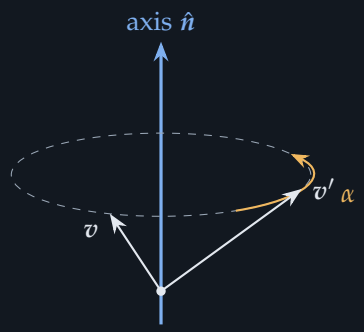


Figure 2.4: One axis, one angle. Turning about the axis $\hat{n}$ by the angle α carries the arrow v around the dashed circle to v'. Any change of orientation can be described this way.

The quaternion stores the same information in a cleverer way. It uses *half* the angle:

$$q = \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix} = \begin{bmatrix} \cos(\alpha/2) \\ \hat{n}\,\sin(\alpha/2) \end{bmatrix} = \begin{bmatrix} \cos(\alpha/2) \\ n_1 \sin(\alpha/2) \\ n_2 \sin(\alpha/2) \\ n_3 \sin(\alpha/2) \end{bmatrix}. \tag{2.25}$$

The first number, q_0, is called the *scalar part*. The other three, $q_v = (q_1, q_2, q_3)$, form the *vector part*; they point along the axis. We explain why the angle is halved in Section 2.6.5. For now, just accept it.

The length is always 1. Add up the squares:

$$q_0^2 + q_1^2 + q_2^2 + q_3^2 = \cos^2\tfrac{\alpha}{2} + \left(n_1^2 + n_2^2 + n_3^2\right)\sin^2\tfrac{\alpha}{2} = \cos^2\tfrac{\alpha}{2} + 1 \cdot \sin^2\tfrac{\alpha}{2} = 1,$$

because $\hat{n}$ has length 1 and $\cos^2 + \sin^2 = 1$. That is why these are called *unit* quaternions. Four numbers with one rule connecting them leaves three free numbers — the same as three angles, but with no point where the description breaks down.

Some quaternions you should recognise.

Rotation	**Half angle**	**Quaternion** (q_0, q_1, q_2, q_3)
no rotation at all	0°	(1, 0, 0, 0)
90° about z	45°	(0.7071, 0, 0, 0.7071)
180° about z	90°	(0, 0, 0, 1)
180° about x	90°	(0, 1, 0, 0)
90° about y (pitch up by 90°)	45°	(0.7071, 0, 0.7071, 0)

Notice the pattern: q_0 is large when the turn is small, and zero for a half turn.

Reading a quaternion back. To get the axis and angle out again, reverse (2.25):

$$\alpha = 2\arccos q_0, \qquad \hat{n} = \frac{q_v}{\sin(\alpha/2)} \quad (\text{when } \alpha \neq 0). \tag{2.26}$$

For example, $\boldsymbol{q} = (0.5,\ 0.5,\ 0.5,\ 0.5)$ gives

$$\alpha = 2\arccos 0.5 = 120^\circ, \qquad \hat{\boldsymbol{n}} = \frac{(0.5,\ 0.5,\ 0.5)}{\sin 60^\circ} = \frac{(1,\ 1,\ 1)}{\sqrt{3}},$$

a 120° turn about the diagonal of a cube. We will meet this one again in a moment.

2.6.3 Step 3: combining two turns (quaternion multiplication)

If a rocket turns one way and then another, the result is a single new turn. With rotation matrices we would multiply the matrices. With quaternions we use the *quaternion product* $\odot$. For two quaternions $\boldsymbol{p} = (p_0, \boldsymbol{p}_v)$ and $\boldsymbol{q} = (q_0, \boldsymbol{q}_v)$:

$$\boldsymbol{p} \odot \boldsymbol{q} = \begin{bmatrix} p_0 q_0 - \boldsymbol{p}_v \cdot \boldsymbol{q}_v \\ p_0\, \boldsymbol{q}_v + q_0\, \boldsymbol{p}_v + \boldsymbol{p}_v \times \boldsymbol{q}_v \end{bmatrix}. \tag{2.27}$$

Read it in words:

- **new scalar part** = (scalar times scalar) minus (dot product of the two vector parts);
- **new vector part** = each scalar times the other's vector part, plus the cross product of the two vector parts.

That is it — just the dot and cross products from Section 2.1.

The order rule. If you turn by $\boldsymbol{q}_1$ first and then by $\boldsymbol{q}_2$, the combined turn is $\boldsymbol{q}_2 \odot \boldsymbol{q}_1$: the first turn goes on the *right*. This is the same as for matrices, where "first $\mathbf{R}_1$, then $\mathbf{R}_2$" is $\mathbf{R}_2\mathbf{R}_1$.

Worked example: Two quarter turns make a half turn. Turn by 90° about z, twice. Both quaternions are $\boldsymbol{q} = (c,\ 0,\ 0,\ s)$ with $c = s = 0.7071$, so $\boldsymbol{q}_v = (0, 0, s)$. Using (2.27):

- scalar part: $c \cdot c - (0,0,s) \cdot (0,0,s) = c^2 - s^2 = 0.5 - 0.5 = 0$;
- vector part: $c\,(0,0,s) + c\,(0,0,s) + (0,0,s) \times (0,0,s) = (0,\ 0,\ 2cs) + \mathbf{0} = (0,\ 0,\ 1)$. (A vector crossed with itself is always zero.)

The result is $(0, 0, 0, 1)$, which our table says is a 180° turn about z. Two quarter turns make a half turn, as they should.

Worked example: Order matters. Now turn 90° about x first, $\boldsymbol{p} = (c,\ s,\ 0,\ 0)$, and then 90° about z, $\boldsymbol{q} = (c,\ 0,\ 0,\ s)$. The combined turn is $\boldsymbol{q} \odot \boldsymbol{p}$:

- scalar part: $c \cdot c - (0,0,s) \cdot (s,0,0) = 0.5 - 0 = 0.5$;
- vector part: $c\,(s,0,0) + c\,(0,0,s) + (0,0,s) \times (s,0,0)$. The cross product is $(0,\ s^2,\ 0) = (0,\ 0.5,\ 0)$, so the vector part is $(0.5,\ 0.5,\ 0.5)$.

So $\boldsymbol{q} \odot \boldsymbol{p} = (0.5,\ 0.5,\ 0.5,\ 0.5)$: the 120° turn about $(1,1,1)/\sqrt{3}$ we decoded earlier. Doing it the other way round gives $\boldsymbol{p} \odot \boldsymbol{q} = (0.5,\ 0.5,\ -0.5,\ 0.5)$, because the cross product changes sign. A different turn: order matters, exactly as we saw with matrices in Section 2.4.

Undoing a turn: the conjugate. To undo a turn, turn back about the same axis by the same angle in the opposite direction. For a quaternion, that just means flipping the sign of the vector part. This is called the *conjugate*:

$$\boldsymbol{q}^* = (q_0,\ -\boldsymbol{q}_v). \tag{2.28}$$

Check with (2.27): the scalar part of $\boldsymbol{q} \odot \boldsymbol{q}^*$ is $q_0^2 + \boldsymbol{q}_v \cdot \boldsymbol{q}_v = 1$, and the vector part is $-q_0\boldsymbol{q}_v + q_0\boldsymbol{q}_v - \boldsymbol{q}_v \times \boldsymbol{q}_v = \mathbf{0}$. So $\boldsymbol{q} \odot \boldsymbol{q}^* = (1, 0, 0, 0)$: no rotation at all.

2.6.4 Step 4: using a quaternion to turn a vector

Now the useful part: given a quaternion $\boldsymbol{q}$ and a vector $\boldsymbol{v}$, where does $\boldsymbol{v}$ end up after the turn? The recipe has three steps.

1. Put the vector into a quaternion with a zero in front: $(0,\ \boldsymbol{v})$.
2. "Sandwich" it: multiply by $\boldsymbol{q}$ on the left and by $\boldsymbol{q}^*$ on the right.
3. The answer is the vector part of the result.

In symbols:

$$\begin{bmatrix} 0 \\ \boldsymbol{v}' \end{bmatrix} = \boldsymbol{q} \odot \begin{bmatrix} 0 \\ \boldsymbol{v} \end{bmatrix} \odot \boldsymbol{q}^*. \tag{2.29}$$

Worked example: Turning the x-axis by 90° about z, step by step. Take $\boldsymbol{q} = (c, 0, 0, s)$ with $c = s = 0.7071$, and $\boldsymbol{v} = (1, 0, 0)$.
First multiplication, $\boldsymbol{q} \odot (0, \boldsymbol{v})$:

- scalar: $c \cdot 0 - (0, 0, s) \cdot (1, 0, 0) = 0$;
- vector: $c\,(1, 0, 0) + 0 + (0, 0, s) \times (1, 0, 0) = (c, 0, 0) + (0, s, 0) = (c,\ s,\ 0)$.

So the middle result is $(0,\ c,\ s,\ 0)$.
Second multiplication, by $\boldsymbol{q}^* = (c,\ 0,\ 0,\ -s)$:

- scalar: $0 \cdot c - (c, s, 0) \cdot (0, 0, -s) = 0$;
- vector: $0 + c\,(c, s, 0) + (c, s, 0) \times (0, 0, -s) = (c^2, cs, 0) + (-s^2, cs, 0) = (c^2 - s^2,\ 2cs,\ 0) = (0,\ 1,\ 0)$.

The x-axis has been turned onto the y-axis: exactly a 90° turn about z.

The shortcut: a rotation matrix. Doing two quaternion products for every vector is slow. If we carry out the sandwich (2.29) once with symbols and collect the terms, we get an ordinary 3×3 matrix that does the same job, $\boldsymbol{v}' = \mathbf{R}(\boldsymbol{q})\,\boldsymbol{v}$:

$$\mathbf{R}(\boldsymbol{q}) = \begin{bmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2(q_1q_2 - q_0q_3) & 2(q_1q_3 + q_0q_2) \\ 2(q_1q_2 + q_0q_3) & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2(q_2q_3 - q_0q_1) \\ 2(q_1q_3 - q_0q_2) & 2(q_2q_3 + q_0q_1) & q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{bmatrix}. \tag{2.30}$$

You do not need to memorise it. Notice the pattern instead: the diagonal has all four squares with different signs, and the off-diagonal entries are pairs $2(q_iq_j \pm q_0q_k)$.

For our example $\boldsymbol{q} = (0.7071, 0, 0, 0.7071)$: $q_0^2 - q_3^2 = 0$, $q_0^2 + q_3^2 = 1$ and $2q_0q_3 = 1$, so

$$\mathbf{R}(\boldsymbol{q}) = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{R}(\boldsymbol{q}) \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix},$$

the same answer as the step-by-step sandwich.

Turning arrows versus relabelling axes. $\mathbf{R}(\boldsymbol{q})$ *moves* an arrow. A direction cosine matrix from Section 2.4 does the opposite job: it keeps the arrow still and *rewrites its numbers* in turned axes. Turning the axes one way looks, from the axes' point of view, like the arrow turning the other way. So if $\boldsymbol{q}$ describes how the body axes are turned relative to the reference axes, the coordinate transformation is the reverse turn, which for a rotation matrix is the transpose:

$$\mathbf{T}^{BA} = \mathbf{R}(\boldsymbol{q})^{\mathsf{T}}. \tag{2.31}$$

> **Reality check**: Check conventions with a known example. Books and software libraries do not all agree on quaternion conventions. Some put the scalar first, some last, and some define the product in the reverse order. None is wrong, but mixing two of them silently turns every rotation backwards. Before trusting a new library, run the 90°-about-z example above through it and check that you get $(0, 1, 0)$.

2.6.5 Why the half angle?

Here is a strange but useful fact. Replace every number in $\boldsymbol{q}$ by its negative. In (2.30), every entry is a product of *two* components, and $(-a)(-b) = ab$, so the matrix does not change at all:

$$\mathbf{R}(-\boldsymbol{q}) = \mathbf{R}(\boldsymbol{q}). \tag{2.32}$$

So $\boldsymbol{q}$ and $-\boldsymbol{q}$ describe exactly the same orientation.

The half angle explains why. Look at what happens to $q_0 = \cos(\alpha/2)$ as you keep turning about one axis:

Turn α	Half angle	q_0	Orientation
0°	0°	$+1$	where we started
180°	90°	0	half turn
360°	180°	-1	back where we started, but $\boldsymbol{q}$ has become $-\boldsymbol{q}$
720°	360°	$+1$	back where we started, and $\boldsymbol{q}$ is back too

Because of the half angle, the quaternion needs *two* full turns to come back to itself, while the orientation needs only one. So every orientation has two quaternions, $\boldsymbol{q}$ and $-\boldsymbol{q}$. Mathematicians call this a *double cover*.

You have seen this before. The qubit state of Chapter 1, $\cos\frac{\vartheta}{2}|0\rangle + e^{i\varphi}\sin\frac{\vartheta}{2}|1\rangle$, also uses half angles, for the same mathematical reason. Unit quaternions and single-qubit gates are two

faces of the same group, which is one more reason the rocket side and the quantum side of this book fit together.

Why this matters in software. When a filter compares two attitude quaternions, say the estimate and a measurement, they might be the same orientation written with opposite signs. Compared naively, they look as if the vehicle has turned by almost 360°. The simple fix: if the scalar part of $q_1^* \odot q_2$ is negative, flip the sign of q_2 before comparing.

2.6.6 How a quaternion changes over time

In flight, the gyroscopes measure the body's angular velocity ω (in body axes, in rad/s). The simulator and the navigation software must turn this into an updated quaternion at every time step. Here is where the formula comes from.

A tiny turn. In a very short time Δt, the body turns about the direction of ω by the small angle $\|\omega\|\,\Delta t$. For a small angle, $\cos(\text{small}/2) \approx 1$ and $\sin(\text{small}/2) \approx \text{small}/2$, so by (2.25) this tiny turn is the quaternion

$$\delta q \approx \begin{bmatrix} 1 \\ \frac{1}{2}\,\omega\,\Delta t \end{bmatrix}. \tag{2.33}$$

Add it on. Because ω is measured in body axes, the tiny turn is applied *after* the current attitude, relative to the body, which puts it on the right: $q(t+\Delta t) = q(t) \odot \delta q$. Then

$$\frac{q(t+\Delta t) - q(t)}{\Delta t} = \frac{q \odot \left[(1,\,\mathbf{0}) + (0,\,\tfrac{1}{2}\omega\Delta t)\right] - q}{\Delta t} = \tfrac{1}{2}\, q \odot \begin{bmatrix} 0 \\ \omega \end{bmatrix},$$

using $q \odot (1, \mathbf{0}) = q$. Letting $\Delta t \to 0$:

$$\boxed{\dot{q} = \tfrac{1}{2}\, q \odot \begin{bmatrix} 0 \\ \omega \end{bmatrix}.} \tag{2.34}$$

Written out in full. Expanding with (2.27) gives four ordinary equations that you can type straight into code:

$$\begin{aligned} \dot{q}_0 &= -\tfrac{1}{2}\,(q_1\omega_1 + q_2\omega_2 + q_3\omega_3), \\ \dot{q}_1 &= \tfrac{1}{2}\,(q_0\omega_1 + q_2\omega_3 - q_3\omega_2), \\ \dot{q}_2 &= \tfrac{1}{2}\,(q_0\omega_2 + q_3\omega_1 - q_1\omega_3), \\ \dot{q}_3 &= \tfrac{1}{2}\,(q_0\omega_3 + q_1\omega_2 - q_2\omega_1). \end{aligned} \tag{2.35}$$

This is exactly the matrix equation $\dot{q} = \frac{1}{2}\blacksquare(\omega)q$ we used in Chapter 1.

> **Worked example**: One time step. The vehicle starts level, $q = (1,0,0,0)$, and pitches up at $\omega = (0,\ 0.1,\ 0)$ rad/s. Then $\dot{q} = (0,\ 0,\ 0.05,\ 0)$. After one step of $\Delta t = 0.01$ s:
>
> $$q \approx (1,\ 0,\ 0.0005,\ 0).$$

The exact answer is a turn of $0.1 \times 0.01 = 0.001$ rad about y, which is

$$(\cos 0.0005,\ 0,\ \sin 0.0005,\ 0) = (0.99999988,\ 0,\ 0.0005,\ 0).$$

The step is very close. The small difference in q_0 is why simulators re-normalise $\boldsymbol{q}$ to length 1 after each step.

Look at the four equations again: there is no division and no tangent anywhere. Nothing can blow up, whichever way the rocket points. That is the whole reason flight software uses quaternions.

2.6.7 Which description to use

Description	Numbers	Good at	Weak at
Euler angles	3	easy for people to read	breaks down at $\theta = \pm 90^\circ$ (gimbal lock)
Direction cosine matrix	9	changing axes quickly	9 numbers for 3 pieces of information; slowly drifts away from a true rotation in a simulation
Quaternion	4	integrating in time; no singularity	less intuitive; $\boldsymbol{q}$ and $-\boldsymbol{q}$ mean the same thing

Table 2.2: Three ways to describe attitude. Flight software usually stores and integrates a quaternion, converts it to a direction cosine matrix when it needs to change axes, and converts it to Euler angles only for display.

2.7 Derivatives when the axes are rotating

2.7.1 The problem

The derivative of a vector depends on the frame you watch it from. The launch-pad example of Section 2.3 showed this: the pad's position does not change as seen from the Earth, but changes at 408 m/s as seen from space. Newton's law $\boldsymbol{F} = m\boldsymbol{a}$ holds only when $\boldsymbol{a}$ is the acceleration seen from an inertial frame. But we usually know the motion relative to the rotating Earth. We need a formula that connects the two.

2.7.2 The transport theorem

Let frame B rotate relative to frame A with angular velocity $\boldsymbol{\omega}^{BA}$. Write a vector using B's axes, $\boldsymbol{v} = \sum_i v_i \hat{\boldsymbol{b}}_i$, and differentiate as seen from A. The product rule gives two kinds of terms:

$$\frac{\mathrm{d}^A \boldsymbol{v}}{\mathrm{d}t} = \underbrace{\sum_i \dot{v}_i \, \hat{\boldsymbol{b}}_i}_{\text{change seen from } B} + \underbrace{\sum_i v_i \, \frac{\mathrm{d}^A \hat{\boldsymbol{b}}_i}{\mathrm{d}t}}_{\text{the axes themselves turning}} . \tag{2.36}$$

The first sum is the derivative an observer riding on B would see, $\mathrm{d}^B \boldsymbol{v}/\,\mathrm{d}t$. For the second, each axis $\hat{\boldsymbol{b}}_i$ is a unit vector fixed in B, so seen from A its tip moves in a circle about $\boldsymbol{\omega}^{BA}$, with velocity $\boldsymbol{\omega}^{BA} \times \hat{\boldsymbol{b}}_i$. (A point at $\boldsymbol{r}$ on a body rotating with $\boldsymbol{\omega}$ moves with velocity $\boldsymbol{\omega} \times \boldsymbol{r}$.) So the second sum is $\sum_i v_i \, \boldsymbol{\omega}^{BA} \times \hat{\boldsymbol{b}}_i = \boldsymbol{\omega}^{BA} \times \boldsymbol{v}$, and

$$\boxed{\frac{\mathrm{d}^A \boldsymbol{v}}{\mathrm{d}t} = \frac{\mathrm{d}^B \boldsymbol{v}}{\mathrm{d}t} + \boldsymbol{\omega}^{BA} \times \boldsymbol{v}.} \tag{2.37}$$

This is the *transport theorem* (Zipfel calls the frame-specific derivative the *rotational time derivative* and builds it as a tensor operator; Appendix D of his book gives the full treatment). It holds for any vector: position, velocity, angular momentum.

2.7.3 Coriolis and centripetal accelerations

Apply (2.37) to the position $\boldsymbol{r}$ of a vehicle, from the inertial frame I and the Earth frame E, with the Earth's constant rotation $\boldsymbol{\omega}_E$:

$$\boldsymbol{v}^I = \boldsymbol{v}^E + \boldsymbol{\omega}_E \times \boldsymbol{r}, \tag{2.38}$$

where $\boldsymbol{v}^I = \mathrm{d}^I \boldsymbol{r}/\,\mathrm{d}t$ and $\boldsymbol{v}^E = \mathrm{d}^E \boldsymbol{r}/\,\mathrm{d}t$. Apply it again to $\boldsymbol{v}^I$, and use $\mathrm{d}^E(\boldsymbol{\omega}_E \times \boldsymbol{r})/\,\mathrm{d}t = \boldsymbol{\omega}_E \times \boldsymbol{v}^E$ because $\boldsymbol{\omega}_E$ is constant:

$$\boldsymbol{a}^I = \frac{\mathrm{d}^E \boldsymbol{v}^I}{\mathrm{d}t} + \boldsymbol{\omega}_E \times \boldsymbol{v}^I \tag{2.39}$$

$$= \left(\boldsymbol{a}^E + \boldsymbol{\omega}_E \times \boldsymbol{v}^E\right) + \boldsymbol{\omega}_E \times \left(\boldsymbol{v}^E + \boldsymbol{\omega}_E \times \boldsymbol{r}\right) \tag{2.40}$$

$$= \boldsymbol{a}^E + \underbrace{2\,\boldsymbol{\omega}_E \times \boldsymbol{v}^E}_{\text{Coriolis}} + \underbrace{\boldsymbol{\omega}_E \times (\boldsymbol{\omega}_E \times \boldsymbol{r})}_{\text{centripetal}} . \tag{2.41}$$

Newton's law uses $\boldsymbol{a}^I$. So if we want to simulate in Earth-relative terms, we must write

$$m\,\boldsymbol{a}^E = \boldsymbol{F} - 2m\,\boldsymbol{\omega}_E \times \boldsymbol{v}^E - m\,\boldsymbol{\omega}_E \times (\boldsymbol{\omega}_E \times \boldsymbol{r}). \tag{2.42}$$

The two extra terms behave like forces, but nothing is pushing; they appear because our reference is turning.

> **Worked example**: How big are they?. *Centripetal*, at the equator: $\omega_E^2 R = (7.292 \times 10^{-5})^2 \times 6.378 \times 10^6 \approx 0.034\,\mathrm{m/s^2}$, about 0.35% of g. Small, but it is part of why measured gravity is weaker at the equator.

> *Coriolis*, for a vehicle moving at 2000 m/s perpendicular to the Earth's axis: $2\omega_E v = 2 \times 7.292 \times 10^{-5} \times 2000 \approx 0.29\,\mathrm{m/s^2}$. That also looks small. But over 100 s it would push the vehicle sideways by about $\tfrac{1}{2}at^2 = \tfrac{1}{2} \times 0.29 \times 100^2 \approx 1.46\,\mathrm{km}$. For a vehicle trying to land on a pad a few tens of metres wide, leaving out the Coriolis term is not an option.

2.8 Tensors and the inertia tensor

2.8.1 What a tensor is

The word *tensor* sounds frightening, but for this book it means something simple: a physical quantity that, like a vector, exists independently of the axes, but whose components are written as a matrix and change in a definite way when the axes change.

- A scalar (mass, temperature) is a tensor of *rank 0*: its value does not depend on the axes at all.
- A vector is a tensor of *rank 1*: three components that transform as $[v]^B = \mathbf{T}^{BA}[v]^A$.
- A *rank-2* tensor relates one vector to another, $\boldsymbol{y} = \mathcal{J}\boldsymbol{x}$, and is written as a 3×3 matrix in each coordinate system.

How do the components of a rank-2 tensor transform? Write the relation in both systems, $[\boldsymbol{y}]^A = [\mathcal{J}]^A[\boldsymbol{x}]^A$ and $[\boldsymbol{y}]^B = [\mathcal{J}]^B[\boldsymbol{x}]^B$. Substitute $[\boldsymbol{y}]^B = \mathbf{T}^{BA}[\boldsymbol{y}]^A$ and $[\boldsymbol{x}]^A = \mathbf{T}^{AB}[\boldsymbol{x}]^B = (\mathbf{T}^{BA})^{\mathsf{T}}[\boldsymbol{x}]^B$:

$$[\boldsymbol{y}]^B = \mathbf{T}^{BA}[\mathcal{J}]^A(\mathbf{T}^{BA})^{\mathsf{T}}[\boldsymbol{x}]^B \quad \Longrightarrow \quad \boxed{[\mathcal{J}]^B = \mathbf{T}^{BA}\,[\mathcal{J}]^A\,(\mathbf{T}^{BA})^{\mathsf{T}}.} \tag{2.43}$$

A transformation matrix on the left and its transpose on the right: that is the signature of a rank-2 tensor.

2.8.2 The inertia tensor

The most important rank-2 tensor in flight dynamics links angular velocity to angular momentum, $\boldsymbol{H} = \mathbf{J}\boldsymbol{\omega}$. For a body made of small masses $\mathrm{d}m$ at positions $\boldsymbol{r} = (x, y, z)$ from the centre of mass,

$$\mathbf{J} = \int \left(\|\boldsymbol{r}\|^2 \mathbf{I} - \boldsymbol{r}\boldsymbol{r}^{\mathsf{T}} \right) \mathrm{d}m = \begin{bmatrix} \int (y^2+z^2)\,\mathrm{d}m & -\int xy\,\mathrm{d}m & -\int xz\,\mathrm{d}m \\ -\int xy\,\mathrm{d}m & \int (x^2+z^2)\,\mathrm{d}m & -\int yz\,\mathrm{d}m \\ -\int xz\,\mathrm{d}m & -\int yz\,\mathrm{d}m & \int (x^2+y^2)\,\mathrm{d}m \end{bmatrix}. \tag{2.44}$$

The diagonal entries are the *moments of inertia*: how hard it is to spin the body about each axis. Mass far from an axis counts a lot (it enters squared). The off-diagonal entries are the *products of inertia*; they are zero when the body is symmetric about the axes.

> **Worked example**: A launch vehicle as a cylinder. Model a fully fuelled vehicle as a uniform solid cylinder of mass $m = 500\,\mathrm{t}$, radius $R = 1.85\,\mathrm{m}$ and length $L = 70\,\mathrm{m}$, with

x along its length. Standard results for a cylinder give

$$J_{xx} = \tfrac{1}{2}mR^2 = \tfrac{1}{2}\times 5\times 10^5 \times 1.85^2 \approx 8.6\times 10^5\,\mathrm{kg\,m^2},$$

$$J_{yy} = J_{zz} = \tfrac{1}{12}m\left(3R^2 + L^2\right) = \tfrac{1}{12}\times 5\times 10^5\times(10.27 + 4900) \approx 2.05\times 10^8\,\mathrm{kg\,m^2}.$$

Pitching the vehicle is about 240 times harder than rolling it. This is why roll can be controlled with small thrusters or by differential gimballing, while pitch and yaw need the main engine's full thrust and a long lever arm.

Principal axes. $\mathbf{J}$ is symmetric and positive definite, so by the spectral theorem there is a set of perpendicular axes in which it is diagonal. These are the *principal axes*, and the diagonal entries are the *principal moments*.

Worked example: Finding principal axes. In a planar problem, suppose $\mathbf{J} = \begin{bmatrix} 10 & 2 \\ 2 & 7 \end{bmatrix}$ (in some unit). The characteristic polynomial is $(10-\lambda)(7-\lambda) - 4 = \lambda^2 - 17\lambda + 66 = 0$, so $\lambda = (17 \pm \sqrt{289 - 264})/2 = (17 \pm 5)/2$, giving principal moments 11 and 6. For $\lambda = 11$: $(10-11)x + 2y = 0$, so $y = x/2$ and the axis is $(2,1)/\sqrt{5}$, at an angle of $\arctan(1/2) \approx 26.6^\circ$. For $\lambda = 6$ the axis is $(-1,2)/\sqrt{5}$, perpendicular to the first, as the theorem promised. Rotating the axes by 26.6° using (2.43) turns $\mathbf{J}$ into $\mathrm{diag}(11, 6)$.

The parallel-axis theorem. As propellant burns, the centre of mass moves, and we need the inertia about a new point. If $\mathbf{J}_C$ is the inertia about the centre of mass and the new reference point is displaced by $\boldsymbol{d}$,

$$\mathbf{J}_O = \mathbf{J}_C + m\left(\|\boldsymbol{d}\|^2\,\mathbf{I} - \boldsymbol{d}\,\boldsymbol{d}^{\mathsf{T}}\right). \tag{2.45}$$

It has the same shape as (2.44), as if all the mass sat at the centre of mass. In Chapter 3 we use this to update $\mathbf{J}(t)$ at every time step.

2.9 Where on Earth? Geodetic coordinates

2.9.1 The Earth is not a sphere

The Earth's rotation makes it bulge at the equator. Navigation systems, including GPS, use the WGS-84 ellipsoid:

$$a = 6\,378\,137\,\mathrm{m}\ \text{(equatorial radius)}, \qquad f = \frac{1}{298.257223563}\ \text{(flattening)}, \qquad e^2 = f(2-f) \approx 0.00669438. \tag{2.46}$$

The polar radius is $a(1-f) \approx 6\,356\,752\,\mathrm{m}$, about 21 km less than the equatorial radius.

A position is given by *geodetic latitude* φ (the angle between the local vertical and the equator plane), longitude λ and height h above the ellipsoid. Because the Earth is flattened, the local vertical does not pass through the Earth's centre. The *geocentric* latitude φ_c (the angle of the

line from the centre) is slightly smaller: $\tan\varphi_c = (1-e^2)\tan\varphi$. The largest difference is about 0.19°, near 45° latitude — roughly 21 km on the ground. Mix the two up and your landing site moves by a city's width.

2.9.2 From latitude, longitude and height to ECEF

Define the *prime vertical radius* $N(\varphi) = a/\sqrt{1-e^2\sin^2\varphi}$. Then the Earth-fixed coordinates are

$$\begin{aligned} x &= (N+h)\cos\varphi\cos\lambda, \\ y &= (N+h)\cos\varphi\sin\lambda, \\ z &= \big(N(1-e^2)+h\big)\sin\varphi. \end{aligned} \tag{2.47}$$

Worked example: Locating a launch pad. Take $\varphi = 28.6083^\circ$, $\lambda = -80.6041^\circ$ (west), $h = 3\,\text{m}$, close to a pad at Cape Canaveral. Then $\sin^2\varphi \approx 0.2293$ and

$$N = \frac{6\,378\,137}{\sqrt{1-0.00669438\times 0.2293}} \approx 6\,383\,037\,\text{m}.$$

Substituting into (2.47):

$$[\boldsymbol{r}]^E \approx (914\,844,\ -5\,528\,577,\ 3\,035\,861)\ \text{m}, \qquad \|\boldsymbol{r}\| \approx 6\,373\,269\,\text{m}.$$

The pad is about 4.9 km closer to the Earth's centre than the equatorial radius, because of the flattening.

2.9.3 From Earth-fixed to local north–east–down

The local-level axes at latitude φ and longitude λ are reached from ECEF by a rotation λ about z, then a rotation of $-(90^\circ+\varphi)$ about the new y. The result is

$$\mathbf{T}^{GE} = \begin{bmatrix} -\sin\varphi\cos\lambda & -\sin\varphi\sin\lambda & \cos\varphi \\ -\sin\lambda & \cos\lambda & 0 \\ -\cos\varphi\cos\lambda & -\cos\varphi\sin\lambda & -\sin\varphi \end{bmatrix}. \tag{2.48}$$

Each row is a local axis written in ECEF coordinates: north, east and down. As a check, transform the Earth's rotation vector $[\boldsymbol{\omega}_E]^E = (0,0,\omega_E)$: the result is $(\omega_E\cos\varphi,\ 0,\ -\omega_E\sin\varphi)$. At the Cape that is $(6.40, 0, -3.49)\times 10^{-5}\,\text{rad/s}$: the Earth's spin seen locally has a northward part and an upward part (negative "down"), exactly as it should in the northern hemisphere.

2.9.4 Gravity, to first order

For now we use the simplest gravity model, a point mass:

$$\boldsymbol{g}(\boldsymbol{r}) = -\frac{\mu}{\|\boldsymbol{r}\|^3}\,\boldsymbol{r}, \qquad \mu = 3.986\,004\,418\times 10^{14}\,\text{m}^3/\text{s}^2. \tag{2.49}$$

At the equatorial radius this gives $9.80\,\text{m/s}^2$; at 200 km altitude, $9.21\,\text{m/s}^2$. Gravity in low orbit is only about 6% weaker than on the ground; astronauts float because they are falling,

not because gravity has gone. In Chapter 4 we add the main correction for the Earth's bulge, the J_2 term.

2.10 Numbers inside a computer

2.10.1 Floating point

Computers store real numbers in *floating point*: a fixed number of significant binary digits and an exponent. Double precision (`float64`, the default in Python and NumPy) keeps about 16 significant decimal digits, with relative spacing $\epsilon \approx 2.2 \times 10^{-16}$. Single precision (`float32`) keeps only about 7.

> **Worked example**: Never store positions in single precision. An ECEF position is around 6.4×10^6 m. In `float32`, numbers between 2^{22} and 2^{23} (about 4.2 and 8.4 million) are spaced exactly 0.5 m apart. So a position stored in single precision can only take values on a half-metre grid — and velocities computed by differencing such positions are garbage. In `float64` the spacing at the same size is about 1×10^{-9} m. Use double precision for every navigation quantity.

2.10.2 Solving linear systems

When you need $\boldsymbol{x}$ from $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, do not compute $\mathbf{A}^{-1}$ and then multiply. Use a solver (`numpy.linalg.solve`), which factorises $\mathbf{A}$ and is both faster and more accurate. For symmetric positive definite matrices such as covariances, the Cholesky factorisation $\mathbf{A} = \mathbf{L}\mathbf{L}^{\mathsf{T}}$ is the standard tool. And keep the condition number in mind: with $\kappa \approx 10^4$ in double precision you lose about 4 of your 16 digits, which is usually fine; with $\kappa \approx 10^{12}$ you have only 4 left.

2.10.3 Trying it in Python

The code below builds the 3-2-1 DCM, checks that it is orthogonal, and checks the quaternion example of Section 2.6 two ways. It needs only NumPy (`pip install numpy`); Labbe's book (Appendix A) explains a full scientific Python setup.

```
import numpy as np

def R1(a): # coordinate transformation, rotation about x
   c, s = np.cos(a), np.sin(a)
   return np.array([[1, 0, 0], [0, c, s], [0, -s, c]])

def R2(a): # about y
   c, s = np.cos(a), np.sin(a)
   return np.array([[c, 0, -s], [0, 1, 0], [s, 0, c]])

def R3(a): # about z
```

```
    c, s = np.cos(a), np.sin(a)
    return np.array([[c, s, 0], [-s, c, 0], [0, 0, 1]])

def dcm_321(psi, theta, phi):
    """T^{BG}: local-level (NED) coordinates -> body coordinates."""
    return R1(phi) @ R2(theta) @ R3(psi)

def quat_mult(p, q):
    p0, pv = p[0], p[1:]
    q0, qv = q[0], q[1:]
    return np.concatenate(([p0*q0 - pv @ qv],
                           p0*qv + q0*pv + np.cross(pv, qv)))

def quat_to_rot(q):
    q0, q1, q2, q3 = q
    return np.array([
        [q0**2+q1**2-q2**2-q3**2, 2*(q1*q2-q0*q3), 2*(q1*q3+q0*q2)],
        [2*(q1*q2+q0*q3), q0**2-q1**2+q2**2-q3**2, 2*(q2*q3-q0*q1)],
        [2*(q1*q3-q0*q2), 2*(q2*q3+q0*q1), q0**2-q1**2-q2**2+q3**2]])

T = dcm_321(np.radians(30), np.radians(80), np.radians(5))
print("orthogonal:", np.allclose(T.T @ T, np.eye(3)), " det =", round(np
    .linalg.det(T), 12))
print("pitch back:", np.degrees(-np.arcsin(T[0, 2])))

q = np.array([np.cos(np.pi/4), 0, 0, np.sin(np.pi/4)]) # 90 deg about z
print("x-axis rotated:", np.round(quat_to_rot(q) @ [1, 0, 0], 12))
v = np.array([0, 1, 0, 0]) # pure quaternion for x
q_conj = q * np.array([1, -1, -1, -1])
print("q * v * q^-1 :", np.round(quat_mult(quat_mult(q, v), q_conj)[1:],
     12))
```

Running it prints:

```
orthogonal: True det = 1.0
pitch back: 79.99999999999999
x-axis rotated: [0. 1. 0.]
q * v * q^-1 : [0. 1. 0.]
```

Note the last digits of "pitch back": 80 comes back as 79.99999999999999. That is floating point at work, an error of about 10^{-14} degrees, and nothing to worry about.

Summary

- A vector is a physical arrow; its components $[v]^A$ depend on the axes. Dot products give angles and projections; cross products give moments, $\boldsymbol{M} = \boldsymbol{r} \times \boldsymbol{F}$.
- For $\dot{\boldsymbol{x}} = \mathbf{A}\boldsymbol{x}$, the eigenvalues of $\mathbf{A}$ are the poles, and $\boldsymbol{x}(t) = e^{\mathbf{A}t}\boldsymbol{x}(0)$. Symmetric matrices have real eigenvalues and perpendicular eigenvectors.

- A *frame* decides what is moving; a *coordinate system* decides which numbers we write. A point on the pad is still in the Earth frame but moves at 408 m/s in the inertial frame.
- Direction cosine matrices change axes, $[\boldsymbol{v}]^B = \mathbf{T}^{BA}[\boldsymbol{v}]^A$; they are orthogonal, so the inverse is the transpose, and they chain as $\mathbf{T}^{CA} = \mathbf{T}^{CB}\mathbf{T}^{BA}$. Order matters.
- Euler angles are intuitive but suffer gimbal lock at $\theta = \pm 90^\circ$, which is exactly where a rocket starts. Quaternions have no singularity; $\boldsymbol{q}$ and $-\boldsymbol{q}$ are the same rotation.
- Derivatives depend on the frame: $\mathrm{d}^A\boldsymbol{v}/\,\mathrm{d}t = \mathrm{d}^B\boldsymbol{v}/\,\mathrm{d}t + \boldsymbol{\omega}^{BA} \times \boldsymbol{v}$. This produces the Coriolis and centripetal terms, which can move a vehicle by kilometres if ignored.
- The inertia tensor is a symmetric rank-2 tensor that transforms as $\mathbf{TJT}^{\mathsf{T}}$; its eigenvectors are the principal axes.
- Positions on Earth use the WGS-84 ellipsoid; geodetic and geocentric latitude differ by up to 0.19°.
- Use double precision for navigation, solvers instead of inverses, and watch the condition number.

Exercises

2.1 Components. A wind of 15 m/s blows toward the north-east. Write it in (north, east) components, and then in the axes of a vehicle heading 60°. Check that the length is unchanged.

2.2 Cross product. Repeat the gimbal-moment example with the engine gimballed in yaw instead: $[\boldsymbol{T}]^B = T(\cos\delta, \sin\delta, 0)$. Which body axis is the moment about, and what is its sign?

2.3 Skew-symmetric matrix. Verify $[\boldsymbol{a}\times]\boldsymbol{b} = \boldsymbol{a} \times \boldsymbol{b}$ for $\boldsymbol{a} = (1,2,3)$ and $\boldsymbol{b} = (4,5,6)$. Then show that $[\boldsymbol{a}\times]\boldsymbol{a} = \mathbf{0}$ for any $\boldsymbol{a}$ and explain it geometrically.

2.4 Eigenvalues. Find the eigenvalues of $\begin{bmatrix} 0 & 1 \\ -6 & -5 \end{bmatrix}$. Is the system $\dot{\boldsymbol{x}} = \mathbf{A}\boldsymbol{x}$ stable?

2.5 Matrix exponential. For $\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ we found $e^{\mathbf{A}\Delta t}$. Now take a constant-acceleration model with state (position, velocity, acceleration) and $\mathbf{A}$ having ones just above the diagonal. Show that $\mathbf{A}^3 = \mathbf{0}$ and find $e^{\mathbf{A}\Delta t}$.

2.6 Orthogonality. Show that $\mathbf{R}_2(\alpha)^{\mathsf{T}}\mathbf{R}_2(\alpha) = \mathbf{I}$ and that $\mathbf{R}_2(\alpha)^{\mathsf{T}} = \mathbf{R}_2(-\alpha)$. Explain why the second fact makes sense.

2.7 Euler angles. Compute $\mathbf{T}^{BG}$ for $\psi = 90^\circ$, $\theta = 45^\circ$, $\phi = 0$. Where does the nose point, in (north, east, down)?

2.8 Gimbal lock in practice. Using (2.24), compute $\dot{\psi}$ when $\theta = 89^\circ$, $\phi = 0$, $q = 0$ and $r = 0.01$ rad/s. Repeat for $\theta = 89.9^\circ$. What happens to a simulator with a fixed time step?

2.9 **Quaternions.** Find the quaternion for a 180° rotation about x. Use (2.30) to find its rotation matrix, and check that it maps $(0, 1, 0)$ to $(0, -1, 0)$.

2.10 **Double cover.** Show directly from (2.30) that $\mathbf{R}(-\boldsymbol{q}) = \mathbf{R}(\boldsymbol{q})$.

2.11 **Coriolis.** A vehicle flies due north at $1500\,\mathrm{m/s}$ at latitude 45°. Using the NED form of $\boldsymbol{\omega}_E$, compute the Coriolis acceleration $-2\boldsymbol{\omega}_E \times \boldsymbol{v}$ in NED coordinates. Which way does it push the vehicle?

2.12 **Inertia.** After burning 80% of its propellant, the vehicle in the cylinder example has mass $150\,\mathrm{t}$. Model it again as a uniform cylinder and recompute J_{yy}. By what factor did it drop, and what does that mean for the gimbal angles needed later in flight?

2.13 **Geodesy.** The Guiana Space Centre in Kourou is at about latitude 5.236°, longitude -52.769°, height $10\,\mathrm{m}$. Convert this to ECEF. Then compute its geocentric latitude.

2.14 **Floating point.** In Python, compare `np.float32(6400000.3)` with `np.float64(6400000.3)`. Explain the result using the spacing argument of Section 2.10.

Reading for this chapter

- P. H. Zipfel, *Modeling and Simulation of Aerospace Vehicle Dynamics*, 2nd ed.: Chapter 2 (mathematical concepts and tensors), Chapter 3 (frames and coordinate systems), Appendix A (matrices) and Appendix D (the rotational time derivative).
- S. Boyd and L. Vandenberghe, *Convex Optimization*: Appendix A.1 (norms) and A.5 (linear algebra), and Appendix C.1 (algorithm complexity).
- M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*: Section 2.1 (linear algebra in quantum notation, including tensor products).
- R. R. Labbe, *Kalman and Bayesian Filters in Python*: Appendix A (installing Python, NumPy and FilterPy) and Appendix B (symbology).
- K. J. Åström and R. M. Murray, *Feedback Systems*: Sections 2.1–2.2 (state-space models), as preparation for Chapter 3.